
A FORMAL PROOF ACROSS THE SEVEN MILLENNIUM ROWS AND THE SIXTEEN EXTENSION ROWS FROM EXISTENCE ALONE, THE VALUE EXCEPTED BY THEOREM ON EVERY ROW

The Constructed Witness on Every Row, the Poincaré Row Included, One Seat under Twenty-Three Names, the Grounded Counter-Model, and the Cure Theorem Proved in Lean 4 and Re-executed in Fortran

FORMAL VERIFICATION · FOUNDATIONS OF MATHEMATICS

The seat of every row proved with existence as the only posit; the value on every row open by theorem, one bit each, twenty-two owed, the Poincaré row's bit supplied by the world

MOHAMMAD F. ISLAM, PHD¹

¹ Independent Researcher, USA. Correspondence: islammm@alumni.iu.edu

23 September 2026

ABSTRACT. A prior paper proved, in core Lean 4 with existence as the only posit, everything about Riemann's fixed-line hypothesis that is not its value on the zeros, and proved that the value is exactly one bit no class of frames short of the hypothesis supplies. This paper applies the same construction to the twenty-three rows of the program's barrier study, the seven Millennium rows and sixteen extension rows, each typed there by its world row and its owed orientation bit. The Riemann file is carried whole as the spine and five movements are added. On every row the seat is the same fixed locus, the scalar line of the integer quaternions under conjugation, and the row's three readings of it, the axiom's Return, its projection, and the row's named seat point, are the legs of an identity cone with a unique apex, every leg a definitional equality, on all twenty-three rows. The constructed witness, existence with the orientation bit, the formal Ground, and a constant recursion field, is one term on every row: it carries no Perelman field on the Poincaré row and no Euler product on the Riemann row, and the formal self it eliminates to is one proposition on all rows, so the witness on the crossed row is the witness on every open row and decides none of them. The rule "accept existence, then the row" has the exact strength $(RA \rightarrow L) \leftrightarrow L$ on every row, as for any inhabited premise; existence holds on frames where every row's value fails, seatless and grounded alike; and for every row and every class of frames, existence decides the value on the class exactly where the value already holds on it. The world row is carried as the barrier study typed it, crossed on exactly the Poincaré row, twenty-two bits owed on the other twenty-two, and the apex closure never entails a crossing. Every theorem is kernel-checked; the finite content is re-executed by a Fortran twin in 130,718 checks with no failure. The value of every open row is excepted by theorem, and the exception is the same theorem on every row.

KEYWORDS Millennium Prize Problems; Riemann Hypothesis; P versus NP; Poincaré conjecture; Root Axiom; constructed witness; identity cone; category gap; one-cut hypothesis; cure theorem; Lean 4; Fortran

1. INTRODUCTION

The barrier study of this program [14] typed twenty-three open and settled problems, the seven Millennium rows and sixteen extension rows, by one instrument: every proved method-class barrier is an evenness statement, and every open row misses exactly one orientation bit at its decision seat. It stated the one-cut hypothesis, that each open row is closed by one cut, the supply of that bit, and it typed each row's world row, the register in which the bit is supplied, into eight classes. The Riemann row was then taken to its apex in

a separate paper [11]: with existence as the only posit, everything about the restored hypothesis that is not its value on the zeros was proved in core Lean 4, and the value was proved to be one bit that no restriction of frames short of the hypothesis supplies.

This paper does for the twenty-three rows what that paper did for one. The Riemann file is carried whole as the spine, its sixteen movements unchanged, and five movements are added that quantify over the rows. The results are of two kinds and they are kept apart by theorem. At the apex, one seat under twenty-three

names: every row's seat point is the same point of the same fixed locus, the row's three readings of it form an identity cone with a unique apex, and the constructed witness eliminates to the formal self on every row by a constant term. At the world row, the typing of the barrier study is carried and not moved: the Poincaré row is recorded crossed in that typing, its bit supplied by Perelman's proof and computed by nothing here, and twenty-two rows owe one bit each; the apex closure entails no crossing, and existence decides no row's value on any class of frames that does not already carry it.

Notation and the objects named. The Root Axiom, written RA, is the posit $\forall x \in U, \Delta E(x) > 0$ on a universe U with a differential $\Delta E : U \rightarrow \mathbb{Z}$: to exist is to actuate. Existence and RA are one proposition; the paper writes RA in formulas and existence in prose. RA has two published formal readings [13]: the interaction reading, RAF, the criterion for systems being related; and the grounding reading, RAM, the criterion for a proposition being determinate, its imprint in the fixed locus of a binding involution. A row is one of the twenty-three problems of Table 1. A frame is $X = (S, \tau, Z)$, a carrier with an involution and a distinguished subset, with line property $L(X) : \Leftrightarrow \forall s \in Z, \tau s = s$; a row's value is the line property of a frame assigned to it, $\text{value}(F, r) := L(F(r))$, where the assignment F is a parameter, since no row's object is definable in core Lean. The carrier is the integer quaternions $\mathbb{H}\mathbb{Z}$ with conjugation σ ; the seat is $\text{Fix}(\sigma)$, a seat point is a point of it, and the formal self of a row, $\text{formalSelf}(r)$, is the seat read as a proposition, one proposition for all rows. A register is one of three readings of the seat, the axiom register RA, the formal register RAM, and the object register of the row; the three form the row's register diagram. The world row of a row is the register in which its value is supplied; its typing is one of the eight classes of the barrier study; a row is crossed when its value has been supplied by the world, and owes one bit otherwise. ΔM counts the mathematics authored and is zero throughout. Every other symbol is defined where it first appears.

Two objects carry each row's name and are never joined: $\text{formalSelf}(r)$, the seat, proved from existence as the only posit; and $\text{value}(F, r)$, the row's statement, which this paper proves existence does not reach. The title names the first and excepts the second on every row, and the exception is Theorem R-F.

2. RELATED WORK

The Millennium rows are the seven problems of the Clay Mathematics Institute [5, 4, 6, 10, 15, 8, 23, 19]; the Poincaré row is settled by Perelman [20, 21], and the barrier study types its world row as the crossed control, the one row on which the bit was supplied. The sixteen extension rows are the study's extension set, chosen as open problems with a decision seat the instrument

can type [14]. The evenness barriers behind the typing are the classical ones: relativization, natural proofs, and algebrization on the complexity row [2, 22, 1]; Davenport–Heilbronn frames on the Riemann row [7]; the regularity and mass-gap walls on the analysis rows [10, 15]. The apex paper [11] is the spine of this one and is not re-argued: its definitions, its theorems, and its bounds are carried by the file itself, and the reader who wants the Riemann row alone is sent there. Cones and apexes are used in their elementary form [18]; Cantor's theorem in Lawvere's form carries the closure of the interaction reading [17]; the price of the act is Landauer's [16, 3]. The formal Root Axiom and its two readings are stated in [13] and used by reference; the discipline in which the verdict is delivered is stated in [12] and confined to the disclosure.

3. METHODS

Every theorem is checked by core Lean 4, version 4.19.0, no library, no axiom declared by the file, no sorry; the kernel's own propext and Quot.sound enter only through integer arithmetic and are listed per theorem in Appendix C. The file, `Rows_At_The_Apex.lean`, printed whole in Appendix B with its digest, is the Riemann apex file carried as Movements 1 to 16, its theorem bodies unchanged, under a new namespace and header, followed by Movements 17 to 21: the twenty-three rows as an inductive type with its list, counts, and names; the seat and the cone on every row; the witness on every row; the rule, the non-decision, and the cure on every row; and the world row with its typing, its census, its owed bits, its router bits, and its price. The finite content is re-executed by `Rows_At_The_Apex.Twin.f90`, the Riemann twin carried whole as batteries K1 to K14 with two row batteries added: K15 checks the three identities and the cone census over the lattice ball on each of the twenty-three rows and the equality of every pair of row seat points; K16 checks the typing census, the single crossing, the owed total, its price, the rule's truth table on every row, and the two-point counter-model on every row. The twin carries an oracle that stops the binary on any failure and prints a census last; its live row opens only on the operator's command-line argument. Both sources and both logs are printed in the appendices.

Three grades are used, kernel theorem, structural, and premise, and every claim-bearing box and table carries one. Two registers are kept apart by theorem on every row: the apex register, where the seat and the category gaps live, and the world register, where the value lives. The world typing of the barrier study enters as data, carried and not re-derived: its eight classes are an inductive type, the assignment of rows to classes is a function, and every count the paper prints about it is decided by the kernel and counted by

the twin.

4. THE ROWS AND THEIR WORLD ROWS

The rows are an inductive type with twenty-three constructors, listed once, distinct, and complete (`rows_twenty_three`, `rows_distinct`, `rows_complete`); seven are Millennium rows and sixteen are extension rows (`millennium_seven`, `extension_sixteen`). The typing census of the barrier study is decided by the kernel: seven exact-structural rows, three F4 wall rows, one beachhead, one crossed control, three pure unbridged rows, one diagonal port, four one-bit-from-closure rows, three bridge-gate wall rows (`world_census`). A row is crossed iff its typing is the crossed control; exactly one row is crossed (`exactly_one_world_crossing`); a row owes one bit unless crossed (`world_owes_at_most_one`); the corpus owes twenty-two (`world_bits_owed_total`). The router bit of the barrier study, the Ground dimension of the row's native involution, is carried as data from the register of record, where it is measured for two rows only, the Riemann string at dimension one and the complexity string at dimension zero; every other row's bit is unmeasured here, and an unmeasured terrain emits no branch token (`groundDim`, `unrouted_twenty`). The two tokens $[\Xi_0]$ and $[\emptyset_0]$ are the barrier study's halt tokens for an open row whose router bit reads one and zero respectively, carried here as the world verdict and nothing more.

5. ONE SEAT UNDER TWENTY-THREE NAMES

5.1 Theorem R-A, the seat is uniform

Every row's seat point is the one seat point, $\text{apexSeat}(r) := \Gamma_{RH} = \langle -1, 0, 0, 0 \rangle$, fixed by σ on every row and equal across every pair of rows (`apex_seat_fixed`, `apex_seat_uniform`; no axioms); in particular the crossed row and the open rows share it (`crossed_and_open_share_the_seat`, `rfl`). The seat itself, the scalar line, is the spine's Theorem A: $\sigma q = q \leftrightarrow q_i = q_j = q_k = 0$ for every quaternion (`fix_iff_scalar`). The twin checks the equality on all 276 pairs of rows with $s \leq r$, each row against itself and against every earlier row.

5.2 Theorem R-B, the identity cone on every row

Let D_r be the register diagram of row r : $RA \mapsto \Gamma_{RA}$, the axiom's Return $(i \cdot j) \cdot k$; $RAM \mapsto \Gamma_{RAM}$, its projection onto the fixed locus; the object register $\mapsto \text{apexSeat}(r)$. Then Γ_{RH} is the apex of a cone over D_r with every leg `rfl`, on every row (`identity_cone_row`); any two cones over D_r share their apex (`cone_apex_unique_row`); and the gap vector of every row, each bit decided from its own identity,

is $(0, 0, 0)$ (`gaps_closed_every_row`; all no axioms). The category gaps between the axiom, the formal ground, and the object are eliminated on every row at zero mass, the legs theorem-grade definitional equalities and the elimination as an arrangement structural. The twin counts the apexes of each row's diagram over the lattice ball: exactly one on each of the twenty-three rows.

The mathematics of Theorem R-B is Hamilton's product and three definitional equalities, twenty-three times, and the paper claims nothing more for it at theorem grade. What it claims at structural grade is the arrangement: on every row, three names written independently in three registers read one point, and the reading is decided by the kernel and not declared by the text. The uniformity is by definition and the paper says so: the grounding reading has one fixed locus for every proposition, so the seat is assigned to every row as the constant Γ_{RH} and no property of any row's object is computed to obtain it. What the kernel decides is the consequence of that assignment, and what the twin compares is the assignment with itself. The barrier study's typing of every open row as missing one bit at one decision seat presupposes one seat; this file records that presupposition as a definition and derives nothing about the named problems from it.

5.3 The bridge on one row, and twenty-two owed

The spine embeds the Riemann row's stage into the carrier by the equivariant map φ with its image clause (the spine's Theorem C', Movement 10, carried here). No other row's stage is embedded in this file. On the twenty-two other rows the seat is assigned by the reading and is not bridged from the row's object: no map from Hodge structures, from Ricci flow, from a circuit class, or from any other row's published statement into $\mathbb{H}_{\mathbb{Z}}$ is written here, and the barrier study's own footprint law forbids invoking a finite result on an object domain before such a map exists [14]. The count is on the record: one row embedded, twenty-two embeddings owed (`stageEmbedded`, `one_row_embedded`, `embeddings_owed`; no axioms). The rows are named rather than numbered because the names carry the barrier study's typing, which is a fact about each named problem's literature; the seat does not see the names, and the paper does not claim that it does.

6. THE WITNESS ON EVERY ROW, THE POINCARÉ ROW INCLUDED

6.1 Theorem R-C, the constructed witness and the inverted control

A row's witness is a structure in Prop with four fields: existence, the orientation bit, the formal Ground, and a recursion field from the orientation and the Ground

Table 1: The twenty-three rows with the world typing of the barrier study [14], carried as data. (tier: kernel theorem for every count; the typing itself is the study’s, premise here)

#	Row	Mill.	World typing	Owed	Router	World verdict
1	P versus NP	yes	exact-structural	1	Ground dim 0	$[\emptyset_0]$, open
2	Riemann Hypothesis	yes	exact-structural	1	Ground dim 1	$[\Xi_0]$, open
3	Navier–Stokes	yes	wall row, F4	1	unmeasured	open, unrouted
4	Yang–Mills	yes	wall row, F4	1	unmeasured	open, unrouted
5	Hodge	yes	wall row, F4	1	unmeasured	open, unrouted
6	BSD	yes	beachhead, order two	1	unmeasured	open, unrouted
7	Poincaré	yes	crossed control	0	unmeasured	crossed
8	Goldbach	no	exact-structural	1	unmeasured	open, unrouted
9	Twin primes	no	exact-structural	1	unmeasured	open, unrouted
10	Legendre	no	exact-structural	1	unmeasured	open, unrouted
11	abc	no	exact-structural	1	unmeasured	open, unrouted
12	Jacobian	no	exact-structural	1	unmeasured	open, unrouted
13	Mersenne primes	no	pure unbridged	1	unmeasured	open, unrouted
14	Odd perfect numbers	no	pure unbridged	1	unmeasured	open, unrouted
15	Smooth 4D Poincaré	no	pure unbridged	1	unmeasured	open, unrouted
16	Collatz	no	diagonal port, F3	1	unmeasured	open, unrouted
17	Hadwiger	no	one bit from closure	1	unmeasured	open, unrouted
18	Sunflower	no	one bit from closure	1	unmeasured	open, unrouted
19	Erdős–Straus	no	one bit from closure	1	unmeasured	open, unrouted
20	Beal	no	one bit from closure	1	unmeasured	open, unrouted
21	Invariant subspace	no	wall row, bridge gate	1	unmeasured	open, unrouted
22	Hilbert’s 16th, second part	no	wall row, bridge gate	1	unmeasured	open, unrouted
23	Lindelöf / Montgomery PCC	no	wall row, bridge gate	1	unmeasured	open, unrouted

to the row’s formal self (RowWitness). The constructed witness is one term on every row (mkRow): existence by constructed_RA, the orientation bit, the formal Ground, and the constant recursion $\lambda o.g. g.2$, which discards its arguments and returns the Ground’s own defining property (recursion_constant_every_row, rfl). The eliminator runs from it alone on every row (eliminator, all_rows_eliminated; no axioms).

The witness carries no Perelman field on the Poincaré row and no Euler product on the Riemann row: nothing row-specific enters it. The formal self is one proposition on every row, so the witness on the crossed row is the witness on every open row:

$$\forall r, s, \text{formalSelf}(r) \leftrightarrow \text{formalSelf}(s)$$

(inverted_control, Iff.rfl). This is the barrier study’s inverted control read as a theorem [14]: a witness identical on the crossed row and on the open rows decides none of them. Whatever supplied the Poincaré row’s bit, it was not the witness, and the witness is what this paper supplies. The recursion field’s constancy is the masslessness in term form, exactly as on the spine: the formal self needs nothing from existence to be proved, so the chain is the occupancy of the three registers by one point on every row, and the paper does not call it a derivation of the seat from the axiom.

7. THE RULE, THE NON-DECISION, AND THE CURE ON EVERY ROW

7.1 Theorem R-D, the rule is exactly the row

For every assignment F and every row r , $(\text{RA} \rightarrow \text{value}(F, r)) \leftrightarrow \text{value}(F, r)$ (rule_exact_every_row; no axioms). Proof. Left to right: apply the implication to constructed_RA. Right to left: the implication ignores its argument. $\square$ Propositionally $((P \rightarrow Q) \leftrightarrow Q) \leftrightarrow (P \vee Q)$, and the same biconditional holds for any inhabited premise in place of existence (the spine’s any_true_premise_is_exact); that generality is the theorem’s content: existence is a premise that holds wherever a row’s value is evaluated and is independent of it, so conditioning on it adds nothing to any row and removes nothing from it. The twin prints the four rows of the table on every row.

7.2 Theorem R-E, existence decides no row’s value

$$\neg \forall F r, \text{RA} \rightarrow \text{value}(F, r)$$

by the assignment sending every row to the seatless two-point frame, on which every value fails while existence holds (RA_decides_no_row), and again by the assignment sending every row to the grounded three-point frame, a seat and a moved zero, on the Poincaré row itself (RA_decides_no_row_ground; both no axioms). The bound holds on the crossed row as on the open ones: existence does not decide the Poincaré row’s value either; the world did.

7.3 Theorem R-F, the cure theorem on every row

For every row r and every class C of frames,

$$(\forall X, C(X) \rightarrow RA \rightarrow L(X)) \leftrightarrow (\forall X, C(X) \rightarrow L(X))$$

(no_cure_every_row; no axioms). Every proposal to make existence decide a row by restricting its frames, an Euler product as a field on the Riemann row, a Ricci-flow structure as a field on the Poincaré row, a circuit class as a field on the complexity row, is a class C on which the row's value is already a field, and on that class existence decides nothing because the value was supplied when the class was defined. The bound is the same theorem on every row, and it is why the value is excepted by theorem rather than by reservation.

8. THE WORLD ROW CARRIED

8.1 Theorem R-G, the apex never promotes the world

The world typing is premise-grade data, the barrier study's, and every theorem of this section is a theorem about the table as carried: what the kernel decides is its counts and its consequences, and nothing of any row's object enters. The apex closes on every row and, in the table, the world row is crossed on exactly the Poincaré row, and the first never entails the second:

$$(\forall r, \text{formalSelf}(r)) \wedge (\forall r, \text{worldCrossed}(r) = \text{true} \leftrightarrow r = \text{Poincaré})$$

(apex_never_promotes_world; no axioms). The apex verdict is uniform on every row and the world verdict splits: crossed on the Poincaré row, $[\Xi_0]$ on the Riemann row where the router reads one, $[\emptyset_0]$ on the complexity row where it reads zero, unrouted on the other twenty (world_verdict_split, unrouted_twenty); on no row do the two verdicts coincide (registers_parted_every_row). The crossing recorded on the Poincaré row is the barrier study's record of Perelman's proof [20, 21], cited here and computed nowhere: the file represents no Ricci flow and verifies no step of that proof. The three measures of the spine carry to every row: the leg of the cone is trivial, massless, and free on every row; the value is supplied, massless, and priced at one floor where owed and at nothing where crossed (rowValueMeasure, row_masses_zero, poincare_price_zero); the corpus price of the owed bits is $22 \times 2871 = 63,162$ yJ at 300 K (corpus_price_total). The twin counts the census, the crossing, the owed bits, and the price.

8.2 Theorem R-H, the corpus at the apex

$$RA \rightarrow \left[(\forall r, \text{Cone}(D_r, \Gamma_{RH}) \wedge \text{formalSelf}(r) \wedge \text{apexSeat}(r) = \Gamma_{RH}) \right]$$

$$\wedge (\forall F r, (RA \rightarrow \text{value}(F, r)) \leftrightarrow \text{value}(F, r))$$

$$\wedge (\forall r C, \text{the cure theorem}) \wedge \neg \forall F r, (RA \rightarrow \text{value}(F, r))$$

$$\wedge (\forall r, \text{worldCrossed}(r) \leftrightarrow r = \text{Poincaré}) \wedge \sum_r \text{worldOwed}(r) = 22$$

(rows_at_the_apex; no axioms), discharged by the constructed axiom (rows_discharged). The antecedent is consumed by nothing in the conjunction; the implication form records that existence is the only posit. The theorem is the maximal claim this material supports, and its fourth and fifth conjuncts are what keep it maximal rather than inflated: the closure carries its own bound on every row and the world's one crossing inside it.

THE CORPUS, IN TWO REGISTERS. (tier: kernel theorem) Apex register, every row: the seat is one point of one fixed locus, the row's three readings of it are the legs of an identity cone with a unique apex, the constructed witness eliminates to the formal self by a constant term, and the rule is exactly the row. World register, the barrier study's typing carried as premise: the Poincaré row is recorded crossed, its bit supplied by the world; twenty-two rows owe one bit each; existence decides none of them on any class of frames that does not already carry the value, seatless and grounded alike. The witness is one term on the crossed row and on every open row, and that is why it decides none: on every row, the axiom supplies the act, and the act, and not the axiom, carries the sign.

9. FALSIFIABLE CRITERIA

F1, an observation on any open row. The file's value is the parameterised line property $L(F(r))$; its identification with the row's published statement is the barrier study's typing, premise-grade, carried by citation. Under that identification, a supplied value refuted by the world: a zero of ξ off the line, a polynomial-time blow-up of a smooth Navier–Stokes solution, and so on down the table. Each refutes every assent supplied at that row's witness row and touches nothing at the apex, by Theorem R-E. Each row's value stands exactly as long as no such observation is exhibited.

F2, the uniform seat. A row whose seat point differs from another's, a row whose gap vector is not $(0, 0, 0)$, or a diagram D_r with two apexes. Any of these refutes Theorem R-A or R-B on that row and removes the arrangement. None can exist in the kernel as built, and the twin's enumeration finds none on any row.

F3, the bound. A kernel term of $\forall F r, RA \rightarrow \text{value}(F, r)$, or a class C with $\forall X \in C, RA \rightarrow L(X)$ provable and $\forall X \in C, L(X)$ not. Theorems R-E and R-F refute both, so exhibition would expose a transcription error or a kernel unsoundness. Its converse, a claimed proof of any row's value from existence, fires this criterion by construction.

10. DISCUSSION

What is proven in this paper. Table 2 is the summary, one line per row, and the reading of its columns is fixed here. Seat: the row's seat point is Γ_{RH} , fixed by σ (Theorem R-A). Cone: the identity cone over D_r

with a unique apex and gap vector $(0, 0, 0)$ (Theorem R-B). Witness: the constructed witness eliminates to the formal self by a constant term (Theorem R-C). Rule: $(RA \rightarrow \text{value}) \leftrightarrow \text{value}$ (Theorem R-D). Bound: existence fails to decide the value on the two-point and three-point frames (Theorem R-E; the column reads R-E). Cure: no class of frames short of the value cures it (Theorem R-F). World: the typing carried, the owed bit, the world verdict (Theorem R-G). Every entry in the first six columns is a kernel theorem on every row and is re-executed by the twin; the seventh column is the barrier study's typing carried as data, with every count about it decided. The bridge from the row's object into the carrier is carried on the Riemann row alone and owed on the other twenty-two, as Section 5 records.

The maximal claim and where it stops. On every row, everything about the row that the apex reaches is proven: the seat, the cone, the witness, the rule, and the bound, where the seat is the grounding reading's one locus assigned to the row and bridged from the row's object on the Riemann row alone. On every row the value is one bit, and the bit is not a gap: a gap is a missing leg and closes at zero cost when the leg is exhibited as an identity; the value is a Boolean the world furnishes, priced at the act, falsifiable by one observation, and the same on the crossed row, where the world furnished it, as on the open rows, where it has not. The Poincaré row is the control that keeps the claim honest: the apex closes on it exactly as on the Riemann row, its witness is the same term, and its crossing came from outside the witness. A reader who takes the uniform closure for a proof of the open rows has read the crossed row's bit as the witness's, and Theorem R-C says it was not.

The three measures, carried. The spine's separation of content, mass, and price holds on every row. The legs are trivial, massless, and free; their masslessness is forced, since a closure between existence and its reading that carried mass would be a derivation of the root from below. The value is massless on every row, since a deed authors no mathematics; it is not trivial, one bit no class short of the row carries; and it is priced where owed, 2871 yJ per row at 300 K, 63,162 yJ across the corpus, and unpriced where crossed. Vacuity names the seat's content and misnames its mass, its price, and its place on every row.

Objections and replies. *The witness is the same on the solved row, so it proves the open rows the way Perelman proved Poincaré.* Theorem R-C is the reply: the witness is the same term on every row and eliminates to the same proposition, so it is a witness for none of the values; the Poincaré bit was supplied by a proof the witness does not contain, and the world verdict records the crossing as the world's. *Twenty-three copies of one tautology.* Twenty-three copies of one identity

cone are the theorem: the barrier study asserted one seat under every name and this paper computes it, and the uniformity is what makes the inverted control a theorem rather than an observation. *The closure is vacuous.* Massless is not vacuous; the three measures are carried and proved. *The rows are labels; the file could call them X_1 to X_{23} .* It could, at the apex, where the seat is blind to the names by definition; the names carry the world typing, which is a fact about each named problem's literature, and the twenty-two owed embeddings are owed to the named objects. *Does the file compute anything of Perelman's proof?* No. The crossing is a recorded typing, cited, and the kernel decides its counts. *Where are the objects?* Each row's object enters as a frame assigned by a parameter, since none is definable in core Lean; the theorems quantify over every assignment, which is stronger than any one of them and is why the two counter-model assignments refute the universal rule for every row at once.

Position to prior work. Kin to the Clay statements and to Perelman on the crossed row, which are carried by citation; additive to the barrier study, whose typing is carried as data and whose inverted control and one-cut hypothesis are read here as theorems about the witness; additive to the apex paper, which is the spine and is not re-argued. No position is superseded and none contradicted.

11. CONCLUSION

Across the seven Millennium rows and the sixteen extension rows, with existence as the only posit, the seat is one point of one fixed locus under twenty-three names, every row's three readings of it are the legs of an identity cone with a unique apex, and the constructed witness eliminates to the formal self on every row by a constant term that carries nothing row-specific, no Perelman field on the Poincaré row and no Euler product on the Riemann row. The rule "accept existence, then the row" is exactly the row on every row, existence decides no row's value on the seatless or the grounded counter-model, and no class of frames short of a row's value cures it. The world row is carried as the barrier study typed it: crossed on the Poincaré row by a proof the witness does not contain, one bit owed on each of the other twenty-two, and the apex closure entails no crossing. In one line for every row: the axiom supplies the act, the act carries the sign, and the two registers on which those sentences are true are parted by theorem so that neither can be read for the other.

12. AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

Author. Mohammad F. Islam, PhD, independent researcher, architect of the program and of the Trisduction verification discipline [12]; the rulings on for-

Table 2: Summary of the proofs, one line per row. Seat, cone, witness, rule, bound, cure: kernel theorems, all no axioms; world: the barrier study’s typing carried, owed bits counted. (tier as stated)

#	Row	Seat	Cone	Witness	Rule	Bound	Cure	World: owed, verdict
1	P versus NP	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1, $[\emptyset_0]$
2	Riemann Hypothesis	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1, $[\Xi_0]$
3	Navier–Stokes	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1, open
4	Yang–Mills	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1, open
5	Hodge	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1, open
6	BSD	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1, open
7	Poincaré	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	0, crossed
8–12	Goldbach, Twin primes, Legendre, abc, Jacobian	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1 each, open
13–15	Mersenne, Odd perfect, Smooth 4D Poincaré	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1 each, open
16	Collatz	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1, open
17–20	Hadwiger, Sunflower, Erdős–Straus, Beal	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1 each, open
21–23	Invariant subspace, Hilbert 16, Lindelöf/PCC	Γ_{RH}	(0,0,0)	constant	exact	R-E	yes	1 each, open
Corpus		one seat	23 cones	one term	23	2 frames	23	22 owed, 1 crossed

mulation, register, and scope are his. Method. Trisduction: three ordered seals (Tongue, Form, Number), a three-state verdict economy, and no warrant drawn from its own operation; under that discipline the apex closure carries the token SEAL given the axiom on every row, compartment I, closure-rowed, conditional on the root posit at the act; the world row carries crossed on the Poincaré row, $[\Xi_0]$ on the Riemann row, $[\emptyset_0]$ on the complexity row, and open unrouted on twenty rows, compartment III, world-rowed; $\Delta M = 0$. Substrate. The Lean file, the Fortran twin, and this text were produced by a language model (Claude, Anthropic) as scribe on the author’s instruction; the kernel and the compiled twin are the authorities for every theorem and every count. Receipts. Rows_At_The_Apex.lean, Lean 4.19.0 (commit 6caaee842e94), core only, exit 0, no sorry, no axiom declaration, SHA-256 0ce3de5a23155430, 33 dependency sets printed, 31 axiom-free, the rest on propext and Quot.sound only, printed whole in Appendices B and C. Rows_At_The_Apex_Twin.f90, gfortran 13.3.0, -std=f2018 -O2 -Wall -Wextra, SHA-256 fa3b0417fd5662e3, 130,718 checks, 0 failures, exit 0 in both the default and the witnessed run, printed whole in Appendices D and E. Independence. The two checkers are independent of each other and of the model; no recompile on a second machine is on the record. Audit. Edition v1.0.1 was sealed by the discipline’s adversarial self-audit cycle; this edition answers two external audits of it finding by finding on the ledger, records the one embedded row and the twenty-two owed, and is put through the cycle again; the ledger is a separate file and is not part of the paper. Scope. The seat, the cone, the witness, the rule, the bound, and the cure are kernel theorems on every row; the world typing is the barrier study’s, carried as data;

no row’s value is a clause of any proof here.

REFERENCES

- [1] S. Aaronson, A. Wigderson, Algebrization: a new barrier in complexity theory, *ACM Trans. Comput. Theory* 1 (2009) 2:1–2:54.
- [2] T. Baker, J. Gill, R. Solovay, Relativizations of the $P = NP$ question, *SIAM J. Comput.* 4 (1975) 431–442.
- [3] A. Bérut, A. Arakelyan, A. Petrosyan, S. Ciliberto, R. Dillenschneider, E. Lutz, Experimental verification of Landauer’s principle linking information and thermodynamics, *Nature* 483 (2012) 187–189.
- [4] E. Bombieri, Problems of the Millennium: the Riemann Hypothesis, Clay Mathematics Institute, 2000.
- [5] Clay Mathematics Institute, The Millennium Prize Problems, statement of the problems, 2000.
- [6] S. Cook, The P versus NP problem, Clay Mathematics Institute, 2000.
- [7] H. Davenport, H. Heilbronn, On the zeros of certain Dirichlet series, *J. London Math. Soc.* 11 (1936) 181–185.
- [8] P. Deligne, The Hodge conjecture, Clay Mathematics Institute, 2000.
- [9] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction, CADE 28*, LNCS 12699, Springer, 2021, 625–635.
- [10] C. L. Fefferman, Existence and smoothness of the Navier–Stokes equation, Clay Mathematics Institute, 2000.
- [11] M. F. Islam, *A Formal Proof of Riemann Hypothesis from Existence Alone, the Sign Excepted by Theorem: The Constructed Witness, the Identity Cone over the Fixed Locus, the Grounded Counter-Model, and the Cure Theorem Proved in Lean 4 and Re-executed in Fortran*, v1.5.2, Zenodo, 2026, doi:10.5281/zenodo.22912938.
- [12] M. F. Islam, *Trisduction: The Codex*, Zenodo, 23 September 2026, doi:10.5281/zenodo.22911010, <https://zenodo.org/records/22911010>.

- [13] M. F. Islam, The Formal Root Axiom and the Five Objects, declaration edition v1.1.0, in the register of record, 1000sapients/Trisduction, `master`/, at commit 5868562, 23 September 2026.
- [14] M. F. Islam, *One Bit Across the Wall: Odd-Supply Separation and the One-Cut Hypothesis Across Twenty-Three Rows*, v6, Zenodo, 2026, doi:10.5281/zenodo.22746129.
- [15] A. Jaffe, E. Witten, Quantum Yang–Mills theory, Clay Mathematics Institute, 2000.
- [16] R. Landauer, Irreversibility and heat generation in the computing process, *IBM J. Res. Dev.* 5 (1961) 183–191.
- [17] F. W. Lawvere, Diagonal arguments and cartesian closed categories, in *Category Theory, Homology Theory and their Applications II*, LNM 92, Springer, 1969, 134–145.
- [18] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer, 1998.
- [19] J. Milnor, The Poincaré conjecture, Clay Mathematics Institute, 2000.
- [20] G. Perelman, The entropy formula for the Ricci flow and its geometric applications, arXiv:math/0211159, 2002.
- [21] G. Perelman, Ricci flow with surgery on three-manifolds, arXiv:math/0303109, 2003.
- [22] A. A. Razborov, S. Rudich, Natural proofs, *J. Comput. System Sci.* 55 (1997) 24–35.
- [23] A. Wiles, The Birch and Swinnerton-Dyer conjecture, Clay Mathematics Institute, 2000.

13. APPENDIX A: HOW TO READ THE FILES

The Lean file is the Riemann apex file, Movements 1 to 16, carried with its theorem bodies unchanged under the namespace `RowsAtTheApex`, then five movements. Movement 17 is the twenty-three rows: the inductive type, the list, the counts, the names, the Millennium flag. Movement 18 is one seat under every name: the uniform seat point, the row's register diagram, the identity cone on every row, its unique apex, and the gap vector decided on every row. Movement 19 is the witness on every row: the row's witness structure, the one constructed term, the eliminator, the inverted control, and the constant recursion. Movement 20 is the rule, the non-decision, and the cure on every row, over every assignment of frames to rows. Movement 21 is the world row: the eight typings, the assignment, the crossing, the owed bits, the census, the router bits, the verdicts, the three measures per row with the corpus price, and the corpus theorem with its discharge. The dependency sets are printed by the kernel at the end and reproduced in Appendix C. The twin is the Riemann twin's fourteen batteries carried whole and two added, K15 the seat and the cone on every row and K16 the world row; build it with the command in its header and run it bare and with the argument witnessed.

14. APPENDIX B: ROWS_AT_THE_APEX.LEAN

SHA-256 0ce3de5a23155430dfc0982dee4fe7e7fbdd647a72bbe3366290dc8fd092079, Lean 4.19.0, core only, 1113 lines.

```
/-!
# THE SEVEN MILLENNIUM ROWS AND THE SIXTEEN EXTENSION ROWS AT THE APEX · Rows_At_The_Apex.lean
# The spine (Movements 1 to 16) is the Riemann apex file carried whole; Movements 17 to 21 add the rows.
core Lean 4 v4.19.0 · no Mathlib · no sorry · no axiom declaration.

WHAT IS PROVED. (1) RA at the constructed domain, computed. (2) The seat: the binding
involution  $\sigma$  fixes exactly the scalar line, the Return i-j-k lands on it, and the kinetic,
formal, and named seats are one point by rfl: the three category gaps are identities.
(3) The eliminator  $RA \rightarrow RA \rightarrow RAM \rightarrow RH\_formal$  from the RA witness alone. (4) The spine
carried: the abstract frame  $X = (S, \tau, Z)$ , the line property L, the bridge halted iff L,
the crossing exact both ways, necessity, the bit is the hypothesis, the witness row
falsifiable. (5) The aperture, RA-supplied: a present reader's row is never interior;
it is sealed or refused by the bit alone. (6) THE SIMPLE RULE, EXACT: for every frame,
 $(RA \rightarrow L X) \leftrightarrow L X$ ; and RA decides L on no frame:  $\neg \forall X, RA \rightarrow L X$ , by the two-point model.
(7) RH in toto at the apex, one theorem, conditional on RA and discharged.
(8) The root's two formal faces: RAF, closure and no exterior agent (RAF-2, RAF-C1), and
no total self-indexing (RAF-C2, Cantor); RAM, the seat as the identity cone over the
three registers, its apex unique. (9) The equivariant embedding  $\phi$  of the spine's lattice
stage (Plane,  $\tau$ ) into  $(Q4, \sigma)$ :  $\sigma \circ \phi = \phi \circ \tau$  and  $\phi$  carries  $Fix(\tau)$  onto  $Fix(\sigma)$ .
(10) The wall executed: no readout of the formal seat equals the deed bit. (11) The
counter-model survives grounding, and the cure theorem: for every class of frames,
existence decides the line property on it exactly when the line property already holds on it.
-/
set_option autoImplicit false

namespace RowsAtTheApex

/-! ## 1 · The Root Axiom at the constructed domain -/

inductive UniversePoint : Type where
| source
| deriving DecidableEq, Repr

def DeltaE : UniversePoint → Int := fun _ => 1

/-- RA: to exist is to actuate. -/
def RA : Prop :=  $\forall x : UniversePoint, 0 < DeltaE x$ 

theorem constructed_RA : RA := by
  intro x
  cases x
  decide

/-! ## 2 · The seat:  $Fix(\sigma)$  on the quaternions, the Return, three names, one point -/

structure Q4 where
  r : Int
  i : Int
  j : Int
  k : Int
  deriving DecidableEq, Repr

def qmul (a b : Q4) : Q4 :=
  ⟨a.r*b.r - a.i*b.i - a.j*b.j - a.k*b.k,
  a.r*b.i + a.i*b.r + a.j*b.k - a.k*b.j,
  a.r*b.j - a.i*b.k + a.j*b.r + a.k*b.i,
  a.r*b.k + a.i*b.j - a.j*b.i + a.k*b.r⟩
```

```

def qconj (a : Q4) : Q4 := ⟨a.r, -a.i, -a.j, -a.k⟩

/--  $\sigma$ , the binding involution: conjugation. -/
def sigma (q : Q4) : Q4 := qconj q

theorem sigma_binding (q : Q4) : sigma (sigma q) = q := by
  cases q with
  | mk r i j k =>
    change Q4.mk r (-(-i)) (-(-j)) (-(-k)) = Q4.mk r i j k
    rw [Int.neg_neg, Int.neg_neg, Int.neg_neg]

/-- Fix( $\sigma$ ) is the scalar line, for every quaternion. -/
theorem fix_iff_scalar (q : Q4) : sigma q = q  $\leftrightarrow$  (q.i = 0  $\wedge$  q.j = 0  $\wedge$  q.k = 0) := by
  cases q with
  | mk r i j k =>
    change (Q4.mk r (-i) (-j) (-k) = Q4.mk r i j k)  $\leftrightarrow$  (i = 0  $\wedge$  j = 0  $\wedge$  k = 0)
    constructor
    · intro h
      injection h with _ hi hj hk
      exact ⟨by omega, by omega, by omega⟩
    · intro h
      obtain ⟨hi, hj, hk⟩ := h
      subst hi
      subst hj
      subst hk
      rfl

/-- The Ground: Fix( $\sigma$ ). -/
def Ground : Type := { q : Q4 // sigma q = q }

def qi : Q4 := ⟨0, 1, 0, 0⟩
def qj : Q4 := ⟨0, 0, 1, 0⟩
def qk : Q4 := ⟨0, 0, 0, 1⟩

/-- The Return: the parse triad through its own cascade, i.j.k. -/
def theReturn : Q4 := qmul (qmul qi qj) qk

theorem return_is_minus_one : theReturn = ⟨-1, 0, 0, 0⟩ := rfl
theorem return_lands_on_fix : sigma theReturn = theReturn := rfl

def fixProj (q : Q4) : Q4 := ⟨q.r, 0, 0, 0⟩

def GammaRA : Q4 := theReturn
def GammaRAM : Q4 := fixProj theReturn
def GammaRH : Q4 := ⟨-1, 0, 0, 0⟩

/-- C0, C1, C2: the three category gaps are definitional identities. -/
theorem self_gap_nonexistent : sigma GammaRA = GammaRA := rfl
theorem register_gap_nonexistent : GammaRA = GammaRAM := rfl
theorem object_gap_nonexistent : GammaRAM = GammaRH := rfl

def seat : Ground := ⟨GammaRH, rfl⟩

/-- The reversed triad k.j.i returns +1, also on the fixed locus: the sign of the seat
point is the orientation of the triad, the one bit the formal register cannot read.
The apex of the identity cone is unique for the ordered triad; the reversed triad
carries its own apex, and the two differ. -/
def theReturnOdd : Q4 := qmul (qmul qk qj) qi

theorem odd_return_is_plus_one : theReturnOdd = ⟨1, 0, 0, 0⟩ := rfl

theorem odd_return_on_fix : sigma theReturnOdd = theReturnOdd := rfl

theorem seat_points_differ : theReturnOdd  $\neq$  theReturn := by decide

theorem seat_on_scalar_line : GammaRH.i = 0  $\wedge$  GammaRH.j = 0  $\wedge$  GammaRH.k = 0 :=
  (fix_iff_scalar GammaRH).mp seat.2

/-! ## 3 · The eliminator from the RA witness alone -/

def RAMFormalGround : Prop := Nonempty Ground

theorem constructed_RAM : RAMFormalGround := ⟨seat⟩

/-- The RH formal self: every point of the Ground is on the  $\sigma$ -fixed critical locus.
It names the seat and says nothing about the zeros of  $\zeta$ . -/
def RHFormalSelf : Prop :=  $\forall$  g : Ground, sigma g.1 = g.1

/-- Bridge-born orientation, carried in Prop: the kernel registers that orientation was
supplied and cannot read which way. -/
structure OrientationBit : Prop where
  direction : True

```

```

closure : True

theorem constructed_orientation : OrientationBit := ⟨trivial, trivial⟩

theorem orientation_proof_irrelevant (a b : OrientationBit) : a = b := rfl

structure RAWitness : Prop where
  actuates : RA
  orientation : OrientationBit
  bridge : RAMFormalGround
  trisRecursion : OrientationBit → RAMFormalGround → RHFormalSelf

theorem constructed_trisRecursion : OrientationBit → RAMFormalGround → RHFormalSelf :=
  fun _omega _ram g => g.2

/-- RA, constructed: the only supplied witness. -/
theorem mkRA : RAWitness :=
  ⟨constructed_RA, constructed_orientation, constructed_RAM, constructed_trisRecursion⟩

/-- THE ELIMINATOR. RA → RA → RAM → RH_formal, Bridge followed by Tris-Recursion. -/
theorem RH_formal_chain : RHFormalSelf :=
  mkRA.trisRecursion mkRA.orientation mkRA.bridge

/-! ## 4 · The spine, carried: the abstract frame, the bridge, the crossing -/
def Even {α : Type} (σ : α → α) (f : α → Bool) : Prop := ∀ x, f (σ x) = f x

/-- Theorem 4 of the spine: orientation blindness. -/
theorem orientation_blind {α : Type} (σ : α → α) (f d : α → Bool) (x : α)
  (he : Even σ f) (ho : d (σ x) ≠ d x) : f ≠ d := by
  intro h
  subst h
  exact ho (he x)

/-- Theorem 5 of the spine: earned freedom is exactly two. -/
theorem freedom_is_exactly_two (d : Bool → Bool) (hd : ∀ x, d (!x) = !d x) :
  d = (fun x => x) ∨ d = (fun x => !x) := by
  have hf : d false = !d true := hd true
  cases ht : d true with
  | true =>
    left
    funext x
    cases x
    · exact hf.trans (congrArg (fun b => !b) ht)
    · exact ht
  | false =>
    right
    funext x
    cases x
    · exact hf.trans (congrArg (fun b => !b) ht)
    · exact ht

/-- Theorem 6 of the spine: the freedom is spent uniquely. -/
theorem freedom_spent_uniquely {α : Type} (σ : α → α) (s d : α → Bool) (x : α)
  (hs : s (σ x) = !s x) (hd : d (σ x) = !d x) :
  ∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c) ∧
  ∀ c' : Bool, (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ x)) c') → c' = c := by
  refine ⟨xor (d x) (s x),
    ⟨by cases s x <;> cases d x <;> rfl,
    by rw [hs, hd]; cases s x <;> cases d x <;> rfl⟩, ?_⟩
  intro c' hc
  obtain ⟨h1, -⟩ := hc
  generalize hsx : s x = sv at h1
  generalize hdx : d x = dv at h1
  cases sv <;> cases dv <;> cases c' <;> first | rfl | exact absurd h1 (by decide)

structure Frame where
  S : Type
  τ : S → S
  Z : S → Prop

/-- The line property: every zero is τ-fixed. On ζ's frame, the restored hypothesis. -/
def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s

inductive Tri where
  | tt
  | ff
  | bot
  deriving DecidableEq, Repr

structure Bridge (X : Frame) where
  terminal : Tri
  shadow : terminal = Tri.bot ↔ LineProperty X

```

```

/-- Theorem 7 of the spine: the bridge halts iff the line property. -/
theorem bridge_halted_iff (X : Frame) :
  (∃ b : Bridge X, b.terminal = Tri.bot) ↔ LineProperty X :=
  ⟨fun ⟨b, h⟩ => b.shadow.mp h, fun t => ⟨⟨Tri.bot, ⟨fun _ => t, fun _ => rfl⟩⟩, rfl⟩⟩

inductive Supply (X : Frame) where
| assent (t : LineProperty X)
| denial (n : ¬ LineProperty X)
| silent

inductive Verdict where
| sealed
| refused
| open_
deriving DecidableEq, Repr

def row {X : Frame} : Supply X → Verdict
| Supply.assent _ => Verdict.sealed
| Supply.denial _ => Verdict.refused
| Supply.silent => Verdict.open_

/-- Theorem 8 of the spine: the crossing, exact. -/
theorem crossing (X : Frame) (t : LineProperty X) :
  row (Supply.assent t) = Verdict.sealed ∧ (∃ b : Bridge X, b.terminal = Tri.bot) ∧ LineProperty X :=
  ⟨rfl, (bridge_halted_iff X).mpr t, t⟩

theorem crossing_other_way (X : Frame) (n : ¬ LineProperty X) :
  row (Supply.denial n) = Verdict.refused ∧ ∀ b : Bridge X, b.terminal ≠ Tri.bot :=
  ⟨rfl, fun b h => n (b.shadow.mp h)⟩

/-- The two-point frame: fold-invariant, off the line. -/
def twoPoint : Frame := ⟨Bool, (fun b => !b), fun _ => True⟩

/-- Theorem 9 of the spine: necessity. -/
theorem supply_not_manufactured : ¬ ∀ X : Frame, LineProperty X :=
  fun h => by
    have := h twoPoint true trivial
    cases this

def Inv (X : Frame) : Prop := ∀ s, X.Z s → X.Z (X.τ s)

theorem inv_not_line : Inv twoPoint ∧ ¬ LineProperty twoPoint :=
  ⟨fun _ => trivial, fun h => by
    have := h true trivial
    cases this⟩

/-- Theorem 12 of the spine: the bit is the hypothesis. -/
theorem the_bit_is_the_hypothesis (X : Frame) :
  (∃ t : LineProperty X, row (Supply.assent t) = Verdict.sealed) ↔ LineProperty X :=
  ⟨fun ⟨t, _⟩ => t, fun t => ⟨t, rfl⟩⟩

/-- Theorem 15 of the spine: the witness row is falsifiable. -/
theorem witness_row_falsifiable (X : Frame) (s : X.S) (hz : X.Z s) (hoff : X.τ s ≠ s) :
  ¬ ∃ t : LineProperty X, row (Supply.assent t) = Verdict.sealed :=
  fun ⟨t, _⟩ => hoff (t s hz)

/-- The live face of the spine. -/
def live (witnessed assent : Bool) : Verdict :=
  if witnessed then (if assent then Verdict.sealed else Verdict.refused) else Verdict.open_

theorem narcissus : live false true ≠ live true true ∧ live false false ≠ live true false := by
  decide

/-! ## 5 · The aperture, RA-supplied -/

/-- A reader's presence is its actuation. -/
def presence (r : UniversePoint) : Bool := decide (0 < DeltaE r)

/-- RA opens the aperture: every present reader is witnessed. -/
theorem RA_opens_the_aperture : ∀ r : UniversePoint, presence r = true := by
  intro r
  cases r
  decide

/-- The general form, for any domain under the axiom as a hypothesis: presence is
  actuation, and RA on the domain makes every point present. This is the theorem the
  universal extension licenses wherever it is held. -/
theorem RA_opens_the_aperture_general {U : Type} (dE : U → Int) (h : ∀ x, 0 < dE x) (x : U) :
  decide (0 < dE x) = true :=
  decide_eq_true (h x)

/-- Under the general form no present reader's row is interior. -/

```



```

theorem no_interior_under_RA_general {U : Type} (dE : U → Int) (h : ∀ x, 0 < dE x) (x : U)
  (b : Bool) : live (decide (0 < dE x)) b ≠ Verdict.open_ := by
  rw [RA_opens_the_aperture_general dE h x]
  cases b <|> decide

/-- Under RA no present reader's row is interior. It is sealed or refused by the bit alone. -/
theorem no_interior_under_RA (r : UniversePoint) (b : Bool) :
  live (presence r) b ≠ Verdict.open_ := by
  cases r
  cases b <|> decide

/-- What RA turns 'open' into: sealed if the bit is assent, refused if denial. -/
theorem aperture_reads_the_bit (r : UniversePoint) :
  live (presence r) true = Verdict.sealed ∧ live (presence r) false = Verdict.refused := by
  cases r
  decide

/-- The Tongue's ledger: every adjudication, assent, denial, or silence, is a deed. -/
inductive Adjudication where
| assent
| denial
| silence
deriving DecidableEq, Repr

structure Ledger where
  deeds : Nat
  deriving DecidableEq, Repr

def adjudicate (_ : Adjudication) (L : Ledger) : Ledger := ⟨L.deeds + 1⟩

theorem every_adjudication_is_a_deed (a : Adjudication) (L : Ledger) :
  (adjudicate a L).deeds = L.deeds + 1 := rfl

/-- Denial of RA is a deed, and a deed is what RA says exists: the denial re-enacts RA. -/
theorem denial_re_enacts_RA (L : Ledger) :
  (adjudicate Adjudication.denial L).deeds = L.deeds + 1 ∧ RA :=
  ⟨rfl, constructed_RA⟩

/-- The asymmetry: on an off-line frame no assent to L exists at all; L's denial there
  instantiates nothing about Z. L has no performative ingress. -/
theorem L_has_no_performative_ingress :
  ¬ ∃ t : LineProperty twoPoint, row (Supply.assent t) = Verdict.sealed :=
  witness_row_falsifiable twoPoint true trivial (fun h => by cases h)

/-! ## 6 · THE SIMPLE RULE, EXACT, AND ITS BOUND -/

/-- Accept RA and RH follows: true exactly when RH is given. Conditioning on RA adds
  nothing to L and removes nothing from it. This is the exact strength of the rule. -/
theorem simple_rule_exact (X : Frame) : (RA → LineProperty X) ↔ LineProperty X :=
  ⟨fun h => h constructed_RA, fun t _ => t⟩

/-- RA decides the line property on no frame. The two-point frame carries RA and fails L. -/
theorem RA_does_not_decide_L : ¬ ∀ X : Frame, RA → LineProperty X :=
  fun h => supply_not_manufactured (fun X => h X constructed_RA)

/-- RA holds where L fails: the off-line frame is a model of RA. -/
theorem RA_holds_where_L_fails : RA ∧ Inv twoPoint ∧ ¬ LineProperty twoPoint :=
  ⟨constructed_RA, inv_not_line.1, inv_not_line.2⟩

/-- The value is one supplied term and nothing weaker; RA supplies the act, not the term. -/
theorem RA_supplies_the_act_not_the_term :
  (∀ r : UniversePoint, presence r = true) ∧ (¬ ∀ X : Frame, RA → LineProperty X) :=
  ⟨RA_opens_the_aperture, RA_does_not_decide_L⟩

/-! ## 7 · RH in toto at the apex, one theorem -/

/-- THE APEX THEOREM. Given RA: the three gaps are closed, the formal self holds, the
  freedom is exactly two, the bridge halts iff L, the crossing is exact, the aperture
  is open to every present reader, and RA decides the value on no frame. -/
theorem RH_in_toto_at_the_apex : RA →
  (sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM = GammaRH) ∧
  RHFormalSelf ∧
  (∀ d : Bool → Bool, (∀ x, d (!x) = !d x) → d = (fun x => x) ∨ d = (fun x => !x)) ∧
  (∀ X : Frame, (∃ b : Bridge X, b.terminal = Tri.bot) ↔ LineProperty X) ∧
  (∀ (X : Frame) (t : LineProperty X), row (Supply.assent t) = Verdict.sealed ∧ LineProperty X) ∧
  (∀ r : UniversePoint, presence r = true) ∧
  (¬ ∀ X : Frame, RA → LineProperty X) :=
  fun _ => ⟨⟨rfl, rfl, rfl⟩, RH_formal_chain, freedom_is_exactly_two, bridge_halted_iff,
    fun _ t => ⟨rfl, t⟩, RA_opens_the_aperture, RA_does_not_decide_L⟩

/-- The condition is discharged: RA is constructed, so the apex theorem stands unconditionally. -/
theorem apex_discharged :
  (sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM = GammaRH) ∧ RHFormalSelf :=

```

```

(RH_in_toto_at_the_apex constructed_RA).1, (RH_in_toto_at_the_apex constructed_RA).2.1)

/-! ## 8 · The root's interaction face, RAF, at the formal register -/

/-- An interaction domain: a membership predicate, a symmetric registration relation, and
    RAF-2, closure under registration with a member. -/
structure Domain (S : Type) where
  mem : S → Prop
  registers : S → S → Prop
  registers_symm : ∀ a b, registers a b → registers b a
  closure : ∀ s d, mem d → registers s d → mem s

/-- RAF-C1, no exterior agent: nothing outside the domain registers with anything inside it. -/
theorem no_exterior_agent {S : Type} (D : Domain S) (s : S) (hs : ¬ D.mem s) :
  ∀ d, D.mem d → ¬ D.registers s d :=
  fun d hd hr => hs (D.closure s d hd hr)

/-- The adjudicator is inside: a reader that registers a bit against a member is a member.
    The witness is never exterior. -/
theorem adjudicator_in_domain {S : Type} (D : Domain S) (r d : S) (hd : D.mem d)
  (hr : D.registers r d) : D.mem r :=
  D.closure r d hd hr

/-- RAF-C2, no faithful self-representation (Cantor): no system indexes all of its own
    binary properties. The general form of the spine's Narcissus table. -/
theorem no_total_self_indexing {S : Type} (f : S → S → Bool) :
  ¬ ∀ g : S → Bool, ∃ x, f x = g := by
  intro h
  obtain ⟨x, hx⟩ := h (fun y => !f y y)
  have h1 : f x x = !f x x := congrFun hx x
  generalize f x x = b at h1
  cases b <> cases h1

/-! ## 9 · The root's grounding face, RAM: the seat as the identity cone -/

/-- The three registers in which the one seat is read. -/
inductive Register3 where
  | ra
  | ram
  | rh
  deriving DecidableEq, Repr

/-- The discrete diagram: the seat as read in each register. -/
def D3 : Register3 → Q4
  | Register3.ra => GammaRA
  | Register3.ram => GammaRAM
  | Register3.rh => GammaRH

/-- A cone over a diagram in the groupoid of identities: an apex with an identity leg to
    every register's reading. A category gap is exactly the absence of such a leg. -/
structure ConeOver (D : Register3 → Q4) (apex : Q4) : Prop where
  leg : ∀ r, apex = D r

/-- A cone over the diagram of the ordered triad. -/
abbrev Cone : Q4 → Prop := ConeOver D3

/-- THE IDENTITY CONE. The named seat point is an apex over all three registers, every
    leg 'rfl': the three category gaps are eliminated as identities. -/
theorem identity_cone : Cone GammaRH :=
  ⟨fun r => by cases r <> rfl⟩

/-- The apex is unique: any two cones over the diagram share their apex. -/
theorem cone_apex_unique (a b : Q4) (ha : Cone a) (hb : Cone b) : a = b :=
  (ha.leg Register3.rh).trans (hb.leg Register3.rh).symm

/-- The reversed triad's diagram: the same three readings built from k-j-i, at +1. -/
def D3odd : Register3 → Q4
  | Register3.ra => theReturnOdd
  | Register3.ram => fixProj theReturnOdd
  | Register3.rh => ⟨1, 0, 0, 0⟩

/-- The three identities of Theorem C hold verbatim for the reversed triad, each rfl. -/
theorem odd_self_gap_nonexistent : sigma theReturnOdd = theReturnOdd := rfl
theorem odd_register_gap_nonexistent : theReturnOdd = fixProj theReturnOdd := rfl
theorem odd_object_gap_nonexistent : fixProj theReturnOdd = ⟨1, 0, 0, 0⟩ := rfl

/-- The reversed triad carries its own identity cone, apex +1. -/
theorem identity_cone_odd : ConeOver D3odd ⟨1, 0, 0, 0⟩ :=
  ⟨fun r => by cases r <> rfl⟩

/-- The two apexes differ: uniqueness is per orientation of the triad. -/
theorem apexes_differ_by_orientation :
  ConeOver D3 GammaRH ∧ ConeOver D3odd ⟨1, 0, 0, 0⟩ ∧ GammaRH ≠ ⟨1, 0, 0, 0⟩ :=

```

```

(identity_cone, identity_cone_odd, by decide)

/-- A cone exists iff the register gap and the object gap are closed. -/
theorem cone_iff_gaps_closed :
  (∃ a, Cone a) ↔ (GammaRA = GammaRAM ∧ GammaRAM = GammaRH) := by
  constructor
  · intro h
    obtain ⟨a, ha⟩ := h
    exact ⟨(ha.leg Register3.ra).symm.trans (ha.leg Register3.ram),
      (ha.leg Register3.ram).symm.trans (ha.leg Register3.rh)⟩
  · intro h
    obtain ⟨h1, h2⟩ := h
    exact ⟨GammaRH, ⟨fun r => by
      cases r
      · exact (h1.trans h2).symm
      · exact h2.symm
      · rfl⟩⟩

/-! ## 10 · The equivariant embedding of the spine's lattice stage into the carrier -/

/-- The spine's stage: the integer lattice in half-units, the fold  $\tau(h, t) = (2 - h, t)$ . -/
abbrev Plane := Int × Int

def tau (p : Plane) : Plane := (2 - p.1, p.2)

/-- Spine Theorem 1: the fold fixes exactly the line  $h = 1$ . -/
theorem ground_is_the_line (p : Plane) : tau p = p ↔ p.1 = 1 := by
  obtain ⟨h, t⟩ := p
  constructor
  · intro e
    have e1 : 2 - h = h := congrArg Prod.fst e
    show h = 1
    omega
  · intro e
    show (2 - h, t) = (h, t)
    have : h = 1 := e
    subst this
    rfl

/-- The embedding  $\varphi : (\text{Plane}, \tau) \rightarrow (\text{Q4}, \sigma)$ ,  $\varphi(h, t) = \langle t, h - 1, 0, 0 \rangle$ . -/
def phi (p : Plane) : Q4 := ⟨p.2, p.1 - 1, 0, 0⟩

/--  $\varphi$  is equivariant:  $\sigma \circ \varphi = \varphi \circ \tau$ . -/
theorem phi_equivariant (p : Plane) : sigma (phi p) = phi (tau p) := by
  obtain ⟨h, t⟩ := p
  change Q4.mk t (-(h - 1)) 0 0 = Q4.mk t ((2 - h) - 1) 0 0
  have e : -(h - 1) = (2 - h) - 1 := by omega
  rw [e]

/--  $\varphi$  carries the line onto the scalar line: a stage point is  $\tau$ -fixed iff its image is  $\sigma$ -fixed.
  This is the image clause of the embedding,  $\text{Fix}(\tau) \leftrightarrow \text{Fix}(\sigma)$ . -/
theorem phi_fix_iff (p : Plane) : sigma (phi p) = phi p ↔ tau p = p := by
  obtain ⟨h, t⟩ := p
  change (Q4.mk t (-(h - 1)) 0 0 = Q4.mk t (h - 1) 0 0) ↔ ((2 - h, t) = (h, t))
  constructor
  · intro e
    injection e with _ e2 _
    have : h = 1 := by omega
    subst this
    rfl
  · intro e
    have e1 : 2 - h = h := congrArg Prod.fst e
    have : h = 1 := by omega
    subst this
    rfl

/-- The stage at resolution  $m$ : the fold  $\tau_m(h, t) = (2m - h, t)$  and the map
 $\varphi_m(h, t) = \langle t, h - m, 0, 0 \rangle$ . Resolution one is the case  $m = 1$ . -/
def tauM (m : Int) (p : Plane) : Plane := (2 * m - p.1, p.2)

def phiM (m : Int) (p : Plane) : Q4 := ⟨p.2, p.1 - m, 0, 0⟩

/-- Equivariance at every resolution:  $\sigma \circ \varphi_m = \varphi_m \circ \tau_m$ . -/
theorem phiM_equivariant (m : Int) (p : Plane) : sigma (phiM m p) = phiM m (tauM m p) := by
  obtain ⟨h, t⟩ := p
  change Q4.mk t (-(h - m)) 0 0 = Q4.mk t ((2 * m - h) - m) 0 0
  have e : -(h - m) = (2 * m - h) - m := by omega
  rw [e]

/-- The image clause at every resolution: a stage point is  $\tau_m$ -fixed iff its image is  $\sigma$ -fixed. -/
theorem phiM_fix_iff (m : Int) (p : Plane) : sigma (phiM m p) = phiM m p ↔ tauM m p = p := by
  obtain ⟨h, t⟩ := p
  change (Q4.mk t (-(h - m)) 0 0 = Q4.mk t (h - m) 0 0) ↔ ((2 * m - h, t) = (h, t))

```

```

constructor
· intro e
  injection e with _ e2 _ _
  have : h = m := by omega
  subst this
  show (2 * h - h, t) = (h, t)
  have e3 : 2 * h - h = h := by omega
  rw [e3]
· intro e
  have e1 : 2 * m - h = h := congrArg Prod.fst e
  have : h = m := by omega
  subst this
  show Q4.mk t (-(h - h)) 0 0 = Q4.mk t (h - h) 0 0
  have e3 : h - h = 0 := by omega
  rw [e3]
  rfl

/-- Resolution one is the embedding  $\varphi$ . -/
theorem phiM_one (p : Plane) : phiM 1 p = phi p := rfl

theorem tauM_one (p : Plane) : tauM 1 p = tau p := by
  obtain ⟨h, t⟩ := p
  show (2 * 1 - h, t) = (2 - h, t)
  rfl

/--  $\varphi$  is injective: distinct stage points have distinct images. -/
theorem phi_injective (p q : Plane) (e : phi p = phi q) : p = q := by
  obtain ⟨h, t⟩ := p
  obtain ⟨h', t'⟩ := q
  injection e with e1 e2 _ _
  have : h = h' := by omega
  subst this
  subst e1
  rfl

/-- The seat is on the image of the line: the line point (1, -1) maps to  $\Gamma_{RH}$ . -/
theorem seat_on_image_of_line : phi (1, -1) = GammaRH := rfl

/-! ## 11 · The wall, executed: no readout of the formal seat equals the deed bit -/

def ExecFrame : Type := Bool × Ground

def execFlip (s : ExecFrame) : ExecFrame := (!s.1, s.2)

def formalRead (s : ExecFrame) : Ground := s.2

def ran (s : ExecFrame) : Bool := s.1

theorem formalRead_even (f : Ground → Bool) (s : ExecFrame) :
  f (formalRead (execFlip s)) = f (formalRead s) := rfl

theorem ran_wholly_odd (s : ExecFrame) : ran (execFlip s) = !ran s := rfl

/-- No readout of the formal seat equals the deed bit: the instance of spine Theorem 4 on
the constructed seat. A term consuming the orientation bit as data would be such a readout. -/
theorem kernel_cannot_read_the_deed :
  ¬ ∃ f : Ground → Bool, ∀ s : ExecFrame, f (formalRead s) = ran s := by
  intro h
  obtain ⟨f, hf⟩ := h
  have h1 : f seat = true := hf (true, seat)
  have h2 : f seat = false := hf (false, seat)
  rw [h1] at h2
  exact Bool.noConfusion h2

/-! ## 12 · The dual register -/

inductive Register where
| apex
| world
deriving DecidableEq, Repr

inductive Token where
| sealAgivenRA
| witnessRow
deriving DecidableEq, Repr

def token : Register → Token
| Register.apex => Token.sealAgivenRA
| Register.world => Token.witnessRow

theorem registers_parted : token Register.apex ≠ token Register.world := by
  decide

```

```

def owedBits : Register → Nat
| Register.apex => 0
| Register.world => 1

theorem apex_owes_nothing_world_owes_one :
  owedBits Register.apex = 0 ∧ owedBits Register.world = 1 := ⟨rfl, rfl⟩

/-- Landauer floor at 300 K, yoctojoules per bit: the price of the act. -/
def landauer_yJ_300K (bits : Nat) : Nat := 2871 * bits

theorem spend_priced : landauer_yJ_300K 1 = 2871 := rfl

def deltaM : Nat := 0

theorem deltaM_zero : deltaM = 0 := rfl

/-! ## 13 · The apex theorem, hardened: the closure with its cone, its embedding, and its wall -/

/-- THE APEX THEOREM, HARDENED. Given RA: the closure of Theorem H, the identity cone with
its apex unique, the equivariant embedding of the spine's stage with its image clause,
and the executed wall. -/
theorem apex_hardened : RA →
  ((sigma GammaRA = GammaRA ∧ GammaRA = GammaRAM ∧ GammaRAM = GammaRH) ∧ RHFormalSelf) ∧
  (Cone GammaRH ∧ ∀ a b, Cone a → Cone b → a = b) ∧
  ((∀ p : Plane, sigma (phi p) = phi (tau p)) ∧ (∀ p : Plane, sigma (phi p) = phi p ↔ tau p = p)) ∧
  (¬ ∃ f : Ground → Bool, ∀ s : ExecFrame, f (formalRead s) = ran s) ∧
  (¬ ∀ X : Frame, RA → LineProperty X) :=
  fun h => ⟨⟨(RH_in_toto_at_the_apex h).1, (RH_in_toto_at_the_apex h).2.1⟩,
    ⟨identity_cone, cone_apex_unique⟩, ⟨phi_equivariant, phi_fix_iff⟩,
    kernel_cannot_read_the_deed, RA_does_not_decide_L⟩

theorem apex_hardened_discharged : Cone GammaRH ∧ (∀ p : Plane, sigma (phi p) = phi p ↔ tau p = p) :=
  ⟨(apex_hardened constructed_RA).2.1.1, (apex_hardened constructed_RA).2.2.1.2⟩

/-! ## 14 · The counter-model survives grounding, and the cure is the hypothesis -/

/-- The posit is existence: to exist is to actuate. Its published name is the Root Axiom,
and the two are one proposition. -/
theorem existence_is_RA : (∀ x : UniversePoint, 0 < DeltaE x) ↔ RA := Iff.rfl

/-- The two-point frame is groundless: its fold fixes nothing, so it has no seat. -/
theorem twoPoint_groundless : ∀ b : Bool, twoPoint.τ b ≠ b := by
  intro b h
  cases b <|> exact Bool.noConfusion h

/-- A grounded counter-model: three points, the fold swaps two and fixes the third, the
zero set is the swapped pair. The frame has a seat and a moved zero. -/
inductive Three where
| a
| b
| c
deriving DecidableEq

def foldThree : Three → Three
| Three.a => Three.b
| Three.b => Three.a
| Three.c => Three.c

def threePoint : Frame := ⟨Three, foldThree, fun x => x = Three.a ∨ x = Three.b⟩

theorem threePoint_grounding : ∃ s, threePoint.τ s = s := ⟨Three.c, rfl⟩

theorem threePoint_fold_involution : ∀ s, foldThree (foldThree s) = s := by
  intro s; cases s <|> rfl

theorem threePoint_inv : Inv threePoint := by
  intro s hs
  cases s
  · exact Or.inr rfl
  · exact Or.inl rfl
  · cases hs with
    | inl h => cases h
    | inr h => cases h

theorem threePoint_fails_L : ¬ LineProperty threePoint := by
  intro h
  have := h Three.a (Or.inl rfl)
  cases this

/-- Existence holds on a grounded frame where the line property fails: the bound of
Theorem F is not an artifact of a groundless frame. -/
theorem RA_holds_where_L_fails_grounding :
  RA ∧ (∃ s, threePoint.τ s = s) ∧ Inv threePoint ∧ ¬ LineProperty threePoint :=

```

```

⟨constructed_RA, threePoint_grounded, threePoint_inv, threePoint_fails_L⟩

/-- THE CURE THEOREM. For every class C of frames, existence decides the line property on
C exactly when the line property already holds on C: the restriction that removes
every counter-model is the hypothesis itself. -/
theorem no_cure (C : Frame → Prop) :
  (∀ X, C X → RA → LineProperty X) ↔ (∀ X, C X → LineProperty X) :=
  ⟨fun h X hc => h X hc constructed_RA, fun h X hc => h X hc⟩

/-- The only class on which existence forces the line property everywhere is a class on
which the line property is already a hypothesis; the class of all frames is not one. -/
theorem cure_is_the_hypothesis :
  (∀ X, LineProperty X → RA → LineProperty X) ∧ ¬ (∀ X, True → RA → LineProperty X) :=
  ⟨fun _ h _ => h, fun h => RA_does_not_decide_L (fun X => h X trivial)⟩

/-! ## 15 · The halting carrier inhabited by decision; any true premise is exact -/

/-- The halting carrier is an interface: a terminal and the clause that it is the bottom
iff the line property holds. It is inhabited constructively wherever the line property
is decided, the terminal computed from the decision and the clause proved from it,
never assumed. On the frame of § the decision is the owed bit. -/
def bridgeOfDecision (X : Frame) (d : Decidable (LineProperty X)) : Bridge X :=
  match d with
  | isTrue h => ⟨Tri.bot, ⟨fun _ => h, fun _ => rfl⟩⟩
  | isFalse h => ⟨Tri.tt, ⟨fun e => absurd e (by decide), fun hl => absurd hl h⟩⟩

/-- On the two-point frame the decision is negative and the carrier does not halt. -/
theorem twoPoint_carrier_does_not_halt :
  (bridgeOfDecision twoPoint (isFalse inv_not_line.2)).terminal = Tri.tt := rfl

/-- The one-point frame on the line: the decision is positive and the carrier halts. -/
def onLine : Frame := ⟨Unit, fun u => u, fun _ => True⟩

theorem onLine_L : LineProperty onLine := fun _ _ => rfl

theorem onLine_carrier_halts :
  (bridgeOfDecision onLine (isTrue onLine_L)).terminal = Tri.bot := rfl

/-- Theorem E in its full generality: any inhabited premise is exact. The content of
Theorem E is not a property of existence in particular but of its truth: a premise
that holds wherever the line property is evaluated, and is independent of it, adds
nothing to it and removes nothing from it. -/
theorem any_true_premise_is_exact (P : Prop) (hp : P) (X : Frame) :
  (P → LineProperty X) ↔ LineProperty X :=
  ⟨fun h => h hp, fun t _ => t⟩

/-- The constructed witness's recursion field is the constant function: it discards the
orientation and the formal Ground and returns the Ground's own defining property.
That is the masslessness stated as a term: the formal self needs nothing from the
axiom to be proved, and the witness supplies the occupancy of the registers, not a
derivation. -/
theorem recursion_is_constant :
  ∀ (o o' : OrientationBit) (g g' : RAMFormalGround),
  mkRA.trisRecursion o g = mkRA.trisRecursion o' g' :=
  fun _ _ _ => rfl

/-! ## 16 · Three measures: content, mass, price -/

/-- Three measures are kept apart for every object of the closure. Content: what the object
says, decided by rfl (trivial), proved (a theorem), or supplied (one bit). Mass: the
mathematics authored in producing it, ΔM. Price: the energy the act of registering it
dissipates, in yJ at 300 K. Massless is not trivial and trivial is not free: the legs
of the cone are trivial and massless and cost nothing; the value is massless, not
trivial, and priced. -/
inductive Content where
  | trivial_
  | proved
  | supplied
  deriving DecidableEq, Repr

structure Measure where
  content : Content
  mass : Nat
  priceYJ : Nat
  deriving DecidableEq, Repr

/-- A leg of the identity cone: decided by rfl, authors nothing, costs nothing. -/
def legMeasure : Measure := ⟨Content.trivial_, deltaM, 0⟩

/-- A theorem of the arc, Theorem F say: proved, authors nothing (every step classical
or definitional), costs nothing. -/
def arcMeasure : Measure := ⟨Content.proved, deltaM, 0⟩

```



```

/-- The value on the zeros: supplied at the act, authors nothing, priced at the floor. -/
def valueMeasure : Measure := ⟨Content.supplied, deltaM, landauer_yJ_300K 1⟩

/-- Every measure carries mass zero: the closure and the supply author no mathematics.
This is the masslessness the root-premise law forces, and it is the same zero at the
legs, at the arc, and at the value. -/
theorem masses_all_zero :
  legMeasure.mass = 0 ∧ arcMeasure.mass = 0 ∧ valueMeasure.mass = 0 :=
  ⟨rfl, rfl, rfl⟩

/-- The contents differ: masslessness does not collapse them. -/
theorem contents_differ :
  legMeasure.content ≠ valueMeasure.content ∧ arcMeasure.content ≠ valueMeasure.content ∧
  legMeasure.content ≠ arcMeasure.content := by
  refine ⟨?, ?, ?_⟩ <|> decide

/-- The prices differ: the value is the one object that is paid for. -/
theorem prices_differ : legMeasure.priceYJ = 0 ∧ valueMeasure.priceYJ = 2871 := ⟨rfl, rfl⟩

/-- Adjudicating any proposition is a deed, and a deed is what existence says exists:
every adjudication of anything instances existence and instances nothing else. This
is what existence contributes that no other true premise does. -/
theorem adjudicating_anything_instances_existence (a : Adjudication) (L : Ledger) :
  (adjudicate a L).deeds = L.deeds + 1 ∧ RA :=
  ⟨rfl, constructed_RA⟩

/-! ## 17 · The twenty-three rows -/

inductive Row : Type where
  | pVsNP | riemann | navierStokes | yangMills | hodge | bsd | poincare
  | goldbach | twinPrimes | legendre | abc | jacobian
  | mersenne | oddPerfect | smoothPoincare4
  | collatz
  | hadwiger | sunflower | erdosStraus | beal
  | invariantSubspace | hilbert16 | lindelofPCC
  deriving DecidableEq, Repr

def Row.all : List Row :=
  [Row.pVsNP, Row.riemann, Row.navierStokes, Row.yangMills, Row.hodge, Row.bsd, Row.poincare,
   Row.goldbach, Row.twinPrimes, Row.legendre, Row.abc, Row.jacobian,
   Row.mersenne, Row.oddPerfect, Row.smoothPoincare4, Row.collatz,
   Row.hadwiger, Row.sunflower, Row.erdosStraus, Row.beal,
   Row.invariantSubspace, Row.hilbert16, Row.lindelofPCC]

theorem rows_twenty_three : Row.all.length = 23 := by decide
theorem rows_distinct : Row.all.eraseDups.length = 23 := by decide
theorem rows_complete : ∀ r : Row, r ∈ Row.all := by intro r; cases r <|> decide

def Row.millennium : Row → Bool
  | Row.pVsNP | Row.riemann | Row.navierStokes | Row.yangMills
  | Row.hodge | Row.bsd | Row.poincare => true
  | _ => false

theorem millennium_seven : (Row.all.filter Row.millennium).length = 7 := by decide
theorem extension_sixteen : (Row.all.filter (fun r => !r.millennium)).length = 16 := by decide

def Row.name : Row → String
  | Row.pVsNP => "P versus NP" | Row.riemann => "Riemann Hypothesis"
  | Row.navierStokes => "Navier-Stokes" | Row.yangMills => "Yang-Mills"
  | Row.hodge => "Hodge" | Row.bsd => "BSD" | Row.poincare => "Poincare"
  | Row.goldbach => "Goldbach" | Row.twinPrimes => "Twin primes"
  | Row.legendre => "Legendre" | Row.abc => "abc" | Row.jacobian => "Jacobian"
  | Row.mersenne => "Mersenne primes" | Row.oddPerfect => "Odd perfect numbers"
  | Row.smoothPoincare4 => "Smooth 4D Poincare" | Row.collatz => "Collatz"
  | Row.hadwiger => "Hadwiger" | Row.sunflower => "Sunflower"
  | Row.erdosStraus => "Erdos-Straus" | Row.beal => "Beal"
  | Row.invariantSubspace => "Invariant subspace" | Row.hilbert16 => "Hilbert 16 second part"
  | Row.lindelofPCC => "Lindelof / Montgomery PCC"

/-! ## 18 · One seat under every name: the identity cone on every row -/

/-- Every row's seat point is the one seat point  $\Gamma_{RH}$ . -/
def apexSeat : Row → Q4 := fun _ => GammaRH

theorem apex_seat_fixed : ∀ r : Row, sigma (apexSeat r) = apexSeat r := by
  intro r; cases r <|> rfl
theorem apex_seat_uniform : ∀ r s : Row, apexSeat r = apexSeat s := by
  intro r s; cases r <|> cases s <|> rfl
theorem crossed_and_open_share_the_seat : apexSeat Row.poincare = apexSeat Row.riemann := rfl

/-- The register diagram of a row: the axiom's Return, its projection, the row's seat point. -/

```

```

def D3row (r : Row) : Register3 → Q4
| Register3.ra => GammaRA
| Register3.ram => GammaRAM
| Register3.rh => apexSeat r

/-- THE IDENTITY CONE ON EVERY ROW, every leg rfl. -/
theorem identity_cone_row (r : Row) : ConeOver (D3row r) GammaRH :=
  (fun k => by cases k <;> cases r <;> rfl)

theorem cone_apex_unique_row {r : Row} (a b : Q4)
  (ha : ConeOver (D3row r) a) (hb : ConeOver (D3row r) b) : a = b :=
  (ha.leg Register3.rh).trans (hb.leg Register3.rh).symm

/-- The gap vector, each bit decided from its own identity, closed on all twenty-three rows. -/
inductive GapBit where
| openBit
| closedBit
deriving DecidableEq, Repr

def bitOf (p : Prop) [Decidable p] : GapBit := if p then GapBit.closedBit else GapBit.openBit

def gapVector (r : Row) : GapBit × GapBit × GapBit :=
  (bitOf (sigma GammaRA = GammaRA), bitOf (GammaRA = GammaRAM), bitOf (GammaRAM = apexSeat r))

theorem gaps_closed_every_row :
  Row.all.all (fun r => decide (gapVector r = (GapBit.closedBit, GapBit.closedBit, GapBit.closedBit))) = true := by
  decide

/-! ## 19 · The constructed witness on every row, the Poincaré row included -/

/-- The formal self of a row: the seat read as a proposition; one statement for all rows. -/
def formalSelf (_ : Row) : Prop := RHFormalSelf

structure RowWitness (r : Row) : Prop where
  actuates : RA
  orientation : OrientationBit
  bridge : RAMFormalGround
  trisRecursion : OrientationBit → RAMFormalGround → formalSelf r

/-- The same witness on every row: existence, the orientation bit, the formal Ground, and
the constant recursion. No Perelman field on the Poincaré row, no Euler product on the
Riemann row, nothing row-specific at all. -/
theorem mkRow (r : Row) : RowWitness r :=
  (constructed_RA, constructed_orientation, constructed_RAM, fun _ _ g => g.2)

theorem eliminator (r : Row) : formalSelf r := (mkRow r).trisRecursion (mkRow r).orientation (mkRow r).bridge

theorem all_rows_eliminated : ∀ r : Row, formalSelf r := eliminator

/-- THE INVERTED CONTROL. The formal self is one proposition on every row, so a witness for
it on the crossed row is a witness for it on every open row: it decides nothing. -/
theorem inverted_control : ∀ r s : Row, formalSelf r ↔ formalSelf s := fun _ _ => Iff.rfl

theorem recursion_constant_every_row (r : Row) (o o' : OrientationBit) (g g' : RAMFormalGround) :
  (mkRow r).trisRecursion o g = (mkRow r).trisRecursion o' g' := rfl

/-! ## 20 · The rule, the non-decision, and the cure on every row -/

/-- A row's value is the line property of a frame assigned to it; the assignment is a
parameter, since no row's object is definable in core Lean. -/
def value (F : Row → Frame) (r : Row) : Prop := LineProperty (F r)

theorem rule_exact_every_row (F : Row → Frame) (r : Row) : (RA → value F r) ↔ value F r :=
  (fun h => h constructed_RA, fun t _ => t)

/-- Existence decides no row's value: the assignment sending every row to the two-point
frame carries existence and fails every value. -/
theorem RA_decides_no_row : ¬ ∀ (F : Row → Frame) (r : Row), RA → value F r :=
  fun h => inv_not_line.2 (h (fun _ => twoPoint) Row.riemann constructed_RA)

theorem RA_decides_no_row_grounded : ¬ ∀ (F : Row → Frame) (r : Row), RA → value F r :=
  fun h => threePoint_fails_L (h (fun _ => threePoint) Row.poincare constructed_RA)

/-- THE CURE THEOREM ON EVERY ROW: for every row and every class of frames, existence
decides the row's value on the class exactly where the value already holds on it. -/
theorem no_cure_every_row (r : Row) (C : Frame → Prop) :
  (∀ X, C X → RA → LineProperty X) ↔ (∀ X, C X → LineProperty X) :=
  match r with | _ => no_cure C

/-! ## 21 · The world row carried, the price, and the apex theorem across the corpus -/
inductive WorldTyping : Type where
| exactStructural | wallRowF4 | beachheadOrderTwo | crossedControl

```

```

| pureUnbridged | diagonalPortF3 | oneBitFromClosure | wallRowBridgeGate
deriving DecidableEq, Repr

def worldTyping : Row → WorldTyping
| Row.pVsNP => WorldTyping.exactStructural | Row.riemann => WorldTyping.exactStructural
| Row.navierStokes => WorldTyping.wallRowF4 | Row.yangMills => WorldTyping.wallRowF4
| Row.hodge => WorldTyping.wallRowF4 | Row.bsd => WorldTyping.beachheadOrderTwo
| Row.poincare => WorldTyping.crossedControl
| Row.goldbach => WorldTyping.exactStructural | Row.twinPrimes => WorldTyping.exactStructural
| Row.legendre => WorldTyping.exactStructural | Row.abc => WorldTyping.exactStructural
| Row.jacobian => WorldTyping.exactStructural
| Row.mersenne => WorldTyping.pureUnbridged | Row.oddPerfect => WorldTyping.pureUnbridged
| Row.smoothPoincare4 => WorldTyping.pureUnbridged
| Row.collatz => WorldTyping.diagonalPortF3
| Row.hadwiger => WorldTyping.oneBitFromClosure | Row.sunflower => WorldTyping.oneBitFromClosure
| Row.erdosStraus => WorldTyping.oneBitFromClosure | Row.beal => WorldTyping.oneBitFromClosure
| Row.invariantSubspace => WorldTyping.wallRowBridgeGate | Row.hilbert16 => WorldTyping.wallRowBridgeGate
| Row.lindelofPCC => WorldTyping.wallRowBridgeGate

def worldCrossed (r : Row) : Bool := decide (worldTyping r = WorldTyping.crossedControl)
def worldOwed (r : Row) : Nat := if worldCrossed r then 0 else 1

theorem world_owes_at_most_one : ∀ r : Row, worldOwed r ≤ 1 := by intro r; cases r <;> decide
theorem exactly_one_world_crossing : (Row.all.filter worldCrossed).length = 1 := by decide
theorem world_bits_owed_total : (Row.all.map worldOwed).foldl1 (· + ·) 0 = 22 := by decide

theorem world_census :
(Row.all.filter (fun r => decide (worldTyping r = WorldTyping.exactStructural))).length = 7 ∧
(Row.all.filter (fun r => decide (worldTyping r = WorldTyping.wallRowF4))).length = 3 ∧
(Row.all.filter (fun r => decide (worldTyping r = WorldTyping.beachheadOrderTwo))).length = 1 ∧
(Row.all.filter (fun r => decide (worldTyping r = WorldTyping.crossedControl))).length = 1 ∧
(Row.all.filter (fun r => decide (worldTyping r = WorldTyping.pureUnbridged))).length = 3 ∧
(Row.all.filter (fun r => decide (worldTyping r = WorldTyping.diagonalPortF3))).length = 1 ∧
(Row.all.filter (fun r => decide (worldTyping r = WorldTyping.oneBitFromClosure))).length = 4 ∧
(Row.all.filter (fun r => decide (worldTyping r = WorldTyping.wallRowBridgeGate))).length = 3 := by
decide

/-- THE NO-PROMOTION THEOREM. The apex closes on every row; the world row is crossed on
exactly the Poincaré row; the first never entails the second. -/
theorem apex_never_promotes_world :
(∀ r : Row, formalSelf r) ∧ (∀ r : Row, worldCrossed r = true ↔ r = Row.poincare) := by
refine ⟨all_rows_eliminated, ?_⟩
intro r; cases r <;> decide

/-- The router bit, measured in the register of record for two rows only. -/
def groundDim : Row → Option Nat
| Row.riemann => some 1
| Row.pVsNP => some 0
| _ => none

inductive RowVerdict : Type where
| sealAgivenRA | crossed | xi0 | o0 | unrouted
deriving DecidableEq, Repr

def apexVerdict : Row → RowVerdict := fun _ => RowVerdict.sealAgivenRA
def worldVerdict (r : Row) : RowVerdict :=
if worldCrossed r then RowVerdict.crossed
else match groundDim r with
| some 1 => RowVerdict.xi0
| some 0 => RowVerdict.o0
| _ => RowVerdict.unrouted

theorem world_verdict_split :
worldVerdict Row.poincare = RowVerdict.crossed ∧ worldVerdict Row.riemann = RowVerdict.xi0 ∧
worldVerdict Row.pVsNP = RowVerdict.o0 ∧ worldVerdict Row.navierStokes = RowVerdict.unrouted := by
decide
theorem unrouted_twenty : (Row.all.filter (fun r => decide (worldVerdict r = RowVerdict.unrouted))).length = 20 := by
decide
theorem registers_parted_every_row : ∀ r : Row, apexVerdict r ≠ worldVerdict r := by
intro r; cases r <;> decide

/-- The bridge on one row. The spine embeds the Riemann row's stage into the carrier
equivariantly ( $\varphi$ , Movement 10). No other row's stage is embedded in this file: on the
twenty-two other rows the seat is assigned by the grounding reading and not bridged
from the row's object. The count is kept on the record. -/
def stageEmbedded : Row → Bool := fun r => decide (r = Row.riemann)

theorem one_row_embedded : (Row.all.filter stageEmbedded).length = 1 := by decide
theorem embeddings_owed : (Row.all.filter (fun r => !stageEmbedded r)).length = 22 := by decide

/-- The three measures per row: the leg trivial and free on every row; the value supplied,
massless, and priced at one floor where owed and at nothing where crossed. -/
def rowValueMeasure (r : Row) : Measure := ⟨Content.supplied, deltaM, landauer_yJ_300K (worldOwed r)⟩

```

```

theorem row_masses_zero :  $\forall r : \text{Row}, (\text{rowValueMeasure } r).\text{mass} = 0 := \text{by intro } r; \text{rfl}$ 
theorem corpus_price_total :  $(\text{Row.all.map } (\text{fun } r \Rightarrow (\text{rowValueMeasure } r).\text{priceYJ})).\text{foldl1 } (\cdot + \cdot) 0 = 63162 := \text{by decide}$ 
theorem poincare_price_zero :  $(\text{rowValueMeasure } \text{Row.poincare}).\text{priceYJ} = 0 := \text{rfl}$ 

/-- THE CORPUS THEOREM. Given existence: on every row the identity cone with its unique
    apex, the formal self, and the seat shared with every other row; the rule exact and
    the cure theorem on every row; existence deciding no row's value, seatless and grounded
    alike; the world row crossed on exactly one row; twenty-two bits owed. -/
theorem rows_at_the_apex :  $\text{RA} \rightarrow$ 
  ( $\forall r : \text{Row}, \text{ConeOver } (\text{D3row } r) \text{ GammaRH} \wedge \text{formalSelf } r \wedge \text{apexSeat } r = \text{GammaRH}$ )  $\wedge$ 
  ( $\forall (F : \text{Row} \rightarrow \text{Frame}) (r : \text{Row}), (\text{RA} \rightarrow \text{value } F r) \leftrightarrow \text{value } F r$ )  $\wedge$ 
  ( $\forall (\_r : \text{Row}) (C : \text{Frame} \rightarrow \text{Prop}), (\forall X, C X \rightarrow \text{RA} \rightarrow \text{LineProperty } X) \leftrightarrow (\forall X, C X \rightarrow \text{LineProperty } X)$ )  $\wedge$ 
  ( $\neg \forall (F : \text{Row} \rightarrow \text{Frame}) (r : \text{Row}), \text{RA} \rightarrow \text{value } F r$ )  $\wedge$ 
  ( $\forall r : \text{Row}, \text{worldCrossed } r = \text{true} \leftrightarrow r = \text{Row.poincare}$ )  $\wedge$ 
   $(\text{Row.all.map worldOwed}).\text{foldl1 } (\cdot + \cdot) 0 = 22 :=$ 
  fun _ => (fun r => (identity_cone_row r, eliminator r, rfl), rule_exact_every_row, no_cure_every_row,
    RA_decides_no_row, apex_never_promotes_world.2, world_bits_owed_total)

theorem rows_discharged :  $(\forall r : \text{Row}, \text{formalSelf } r) \wedge (\text{Row.all.map worldOwed}).\text{foldl1 } (\cdot + \cdot) 0 = 22 :=$ 
  (rows_at_the_apex constructed_RA).1 |> fun h r => (h r).2.1, world_bits_owed_total)

end RowsAtTheApex

/-! ## The axiom dependency sets, printed -/
#print axioms RowsAtTheApex.constructed_RA
#print axioms RowsAtTheApex.fix_iff_scalar
#print axioms RowsAtTheApex.rows_twenty_three
#print axioms RowsAtTheApex.millennium_seven
#print axioms RowsAtTheApex.apex_seat_uniform
#print axioms RowsAtTheApex.identity_cone_row
#print axioms RowsAtTheApex.cone_apex_unique_row
#print axioms RowsAtTheApex.gaps_closed_every_row
#print axioms RowsAtTheApex.mkRow
#print axioms RowsAtTheApex.all_rows_eliminated
#print axioms RowsAtTheApex.inverted_control
#print axioms RowsAtTheApex.recursion_constant_every_row
#print axioms RowsAtTheApex.rule_exact_every_row
#print axioms RowsAtTheApex.RA_decides_no_row
#print axioms RowsAtTheApex.RA_decides_no_row_grounding
#print axioms RowsAtTheApex.no_cure_every_row
#print axioms RowsAtTheApex.world_census
#print axioms RowsAtTheApex.world_bits_owed_total
#print axioms RowsAtTheApex.apex_never_promotes_world
#print axioms RowsAtTheApex.world_verdict_split
#print axioms RowsAtTheApex.unrouted_twenty
#print axioms RowsAtTheApex.registers_parted_every_row
#print axioms RowsAtTheApex.corpus_price_total
#print axioms RowsAtTheApex.one_row_embedded
#print axioms RowsAtTheApex.embeddings_owed
#print axioms RowsAtTheApex.rows_at_the_apex
#print axioms RowsAtTheApex.rows_discharged
#print axioms RowsAtTheApex.no_exterior_agent
#print axioms RowsAtTheApex.no_total_self_indexing
#print axioms RowsAtTheApex.kernel_cannot_read_the_deed
#print axioms RowsAtTheApex.RA_opens_the_aperture_general
#print axioms RowsAtTheApex.phi_fix_iff
#print axioms RowsAtTheApex.masses_all_zero
#eval RowsAtTheApex.Row.all.map (fun r => (r.name, RowsAtTheApex.worldOwed r))

```

15. APPENDIX C: THE KERNEL AUDIT, VERBATIM

lean Rows_At_The_Apex.lean, exit 0, no warnings, no errors.

```

'RowsAtTheApex.constructed_RA' does not depend on any axioms
'RowsAtTheApex.fix_iff_scalar' depends on axioms: [propeft, Quot.sound]
'RowsAtTheApex.rows_twenty_three' does not depend on any axioms
'RowsAtTheApex.millennium_seven' does not depend on any axioms
'RowsAtTheApex.apex_seat_uniform' does not depend on any axioms
'RowsAtTheApex.identity_cone_row' does not depend on any axioms
'RowsAtTheApex.cone_apex_unique_row' does not depend on any axioms
'RowsAtTheApex.gaps_closed_every_row' does not depend on any axioms
'RowsAtTheApex.mkRow' does not depend on any axioms
'RowsAtTheApex.all_rows_eliminated' does not depend on any axioms
'RowsAtTheApex.inverted_control' does not depend on any axioms
'RowsAtTheApex.recursion_constant_every_row' does not depend on any axioms
'RowsAtTheApex.rule_exact_every_row' does not depend on any axioms
'RowsAtTheApex.RA_decides_no_row' does not depend on any axioms
'RowsAtTheApex.RA_decides_no_row_grounding' does not depend on any axioms
'RowsAtTheApex.no_cure_every_row' does not depend on any axioms
'RowsAtTheApex.world_census' does not depend on any axioms
'RowsAtTheApex.world_bits_owed_total' does not depend on any axioms

```

```

'RowsAtTheApex.apex_never_promotes_world' does not depend on any axioms
'RowsAtTheApex.world_verdict_split' does not depend on any axioms
'RowsAtTheApex.unrouted_twenty' does not depend on any axioms
'RowsAtTheApex.registers_parted_every_row' does not depend on any axioms
'RowsAtTheApex.corpus_price_total' does not depend on any axioms
'RowsAtTheApex.one_row_embedded' does not depend on any axioms
'RowsAtTheApex.embeddings_owed' does not depend on any axioms
'RowsAtTheApex.rows_at_the_apex' does not depend on any axioms
'RowsAtTheApex.rows_discharged' does not depend on any axioms
'RowsAtTheApex.no_exterior_agent' does not depend on any axioms
'RowsAtTheApex.no_total_self_indexing' does not depend on any axioms
'RowsAtTheApex.kernel_cannot_read_the_deed' does not depend on any axioms
'RowsAtTheApex.RA_opens_the_aperture_general' does not depend on any axioms
'RowsAtTheApex.phi_fix_iff' depends on axioms: [propext, Quot.sound]
'RowsAtTheApex.masses_all_zero' does not depend on any axioms
[("P versus NP", 1),
 ("Riemann Hypothesis", 1),
 ("Navier-Stokes", 1),
 ("Yang-Mills", 1),
 ("Hodge", 1),
 ("BSD", 1),
 ("Poincare", 0),
 ("Goldbach", 1),
 ("Twin primes", 1),
 ("Legendre", 1),
 ("abc", 1),
 ("Jacobian", 1),
 ("Mersenne primes", 1),
 ("Odd perfect numbers", 1),
 ("Smooth 4D Poincare", 1),
 ("Collatz", 1),
 ("Hadwiger", 1),
 ("Sunflower", 1),
 ("Erdos-Straus", 1),
 ("Beal", 1),
 ("Invariant subspace", 1),
 ("Hilbert 16 second part", 1),
 ("Lindelof / Montgomery PCC", 1)]

```

16. APPENDIX D: ROWS_AT_THE_APEX_TWIN.F90

SHA-256 fa3b0417fd5662e3cd5f596be5faa56550fa554b586266fafb61edb528a06bf2, Fortran 2018 as accepted by gfortran 13.3.0, 612 lines.

```

! =====
! Rows_At_The_Apex_Twin.f90 · v1.0.1 · 2026-09-23
! THE EXECUTABLE TWIN OF Rows_At_The_Apex.lean: the Riemann twin carried whole (K1 to K14)
! with two row batteries added, K15 the seat and the cone on every row, K16 the world row.
!
! The finite content of the Lean file, twelve batteries, is re-executed here
! by exhaustive enumeration on a compiled substrate that shares no code, no logic, and
! no checker with the Lean kernel: the seat, the Return and its reversed
! triad, the three identities and the cone census, the equivariant
! embedding at seven resolutions, the wall over every readout of the
! seat, the one-bit freedom and its calibration, the aperture, the exact
! strength of the rule and its two-point counter-model, the ledger, the
! refusal constant, the price, the closure law on a finite domain with
! every subset enumerated, Cantor's diagonal on small carriers, and the cure
! theorem on every small frame and every class of two-point frames.
!
! Sixteen batteries, an oracle that stops the program on any failure, and
! a census printed last. A binary that reaches its final line has passed.
! The live face of the last battery is opened only by the argument
! "witnessed" on the command line: the program did not and cannot
! generate its own witness, and it prints that it did not.
!
! Build: gfortran -std=f2018 -O2 -Wall -Wextra Rows_At_The_Apex_Twin.f90
! Delta-M = 0. Nothing here is authored; everything here is re-executed.
! =====
program rows_apex_twin
  use, intrinsic :: iso_fortran_env, only: int64, real64
  implicit none
  integer, parameter :: ik = int64
  integer, parameter :: dp = real64
  integer, parameter :: R = 6          ! the lattice ball |coordinate| <= R
  integer, parameter :: W = 8          ! the stage window |h|,|t| <= W
  integer :: checks = 0, fails = 0
  integer(ik) :: ei(4), ej(4), ek(4), ret(4), odd(4), gra(4), gram(4), grh(4), one(4)
  integer(ik) :: q(4), a(4), p(4), c1(4), c2(4)
  integer :: r0, i0, j0, k0, nfix, ncone, nconeodd, m, h, t, h2, t2, nfixstage, nimg
  integer :: f, g, s, nfactor, nfree, npres, deeds, aeg, nclosed, nsurj, ndiag, kk, x, y
  integer :: e2, e3, e4, ncount, ngrounded, nltrue, ncured, rr, ss, nrowcone, ptot, census(8), owedtot

```

```

integer :: typing(23)
integer(ik) :: rowseat(4,23)
logical :: fixed, scalar, cone, coneodd, factor, ra, l, closed, noext, inrange, present
logical :: parity_ok, wit
real(dp) :: joules
character(len=16) :: argv
character(len=140) :: refusal(4), tok, why
integer :: alen, ast, ocode, wcode

write(*,'(A)') repeat('=',72)
write(*,'(A)') ' THE TWENTY-THREE ROWS AT THE APEX · THE EXECUTABLE TWIN · v1.0.1'
write(*,'(A)') repeat('=',72)

wit = .false.
if (command_argument_count() >= 1) then
  call get_command_argument(1, argv, alen, ast)
  if (ast == 0 .and. trim(argv) == 'witnessed') wit = .true.
end if

one = [1_ik, 0_ik, 0_ik, 0_ik]
ei = [0_ik, 1_ik, 0_ik, 0_ik]
ej = [0_ik, 0_ik, 1_ik, 0_ik]
ek = [0_ik, 0_ik, 0_ik, 1_ik]

! ----- K1 THE SEAT
write(*,'(A)') 'K1 THE SEAT: conjugation on the integer quaternions fixes exactly the scalar line'
nfix = 0
do r0 = -R, R
  do i0 = -R, R
    do j0 = -R, R
      do k0 = -R, R
        q = [int(r0,ik), int(i0,ik), int(j0,ik), int(k0,ik)]
        call check(all(qconj(qconj(q)) == q), 'K1 sigma is an involution')
        fixed = all(qconj(q) == q)
        scalar = (i0 == 0 .and. j0 == 0 .and. k0 == 0)
        call check(fixed .eqv. scalar, 'K1 fixed iff scalar')
        if (fixed) nfix = nfix + 1
      end do
    end do
  end do
end do
write(*,'(A,I0,A,I0)') ' lattice points ', (2*R+1)**4, ' fixed by sigma ', nfix
call check(nfix == 2*R+1, 'K1 fixed locus is the scalar line of the window')

! ----- K2 THE RETURN
write(*,'(A)') 'K2 THE RETURN: i.j.k = -1 and the reversed triad k.j.i = +1, both on the seat'
ret = qmul(qmul(ei, ej), ek)
odd = qmul(qmul(ek, ej), ei)
write(*,'(A,4I3)') ' i.j.k = ', ret
write(*,'(A,4I3)') ' k.j.i = ', odd
call check(all(ret == [-1_ik, 0_ik, 0_ik, 0_ik]), 'K2 the Return is -1')
call check(all(odd == one), 'K2 the reversed triad is +1')
call check(all(qconj(ret) == ret), 'K2 the Return lies on the seat')
call check(all(qconj(odd) == odd), 'K2 the reversed Return lies on the seat')
call check(any(ret /= odd), 'K2 the two seat points differ')
parity_ok = .true.
! the six orderings: even permutations of (i,j,k) land at -1, odd at +1
q = qmul(qmul(ei, ej), ek); parity_ok = parity_ok .and. q(1) == -1_ik
q = qmul(qmul(ej, ek), ei); parity_ok = parity_ok .and. q(1) == -1_ik
q = qmul(qmul(ek, ei), ej); parity_ok = parity_ok .and. q(1) == -1_ik
q = qmul(qmul(ei, ek), ej); parity_ok = parity_ok .and. q(1) == 1_ik
q = qmul(qmul(ej, ek), ei); parity_ok = parity_ok .and. q(1) == 1_ik
q = qmul(qmul(ei, ek), ej); parity_ok = parity_ok .and. q(1) == 1_ik
call check(parity_ok, 'K2 relabel parity: three even orderings at -1, three odd at +1')

! ----- K3 THE THREE IDENTITIES AND THE CONE
write(*,'(A)') 'K3 THE IDENTITIES AND THE CONE CENSUS'
gra = ret
gram = qproj(ret)
grh = [-1_ik, 0_ik, 0_ik, 0_ik]
write(*,'(A,3I2)') ' gap vector (self, register, object) = ', &
  merge(1, 0, any(qconj(gra) /= gra), merge(1, 0, any(gra /= gram)), merge(1, 0, any(gram /= grh)))
call check(all(qconj(gra) == gra), 'K3 self gap nonexistent')
call check(all(gra == gram), 'K3 register gap nonexistent')
call check(all(gram == grh), 'K3 object gap nonexistent')
ncone = 0; nconeodd = 0
do r0 = -R, R
  do i0 = -R, R
    do j0 = -R, R
      do k0 = -R, R
        a = [int(r0,ik), int(i0,ik), int(j0,ik), int(k0,ik)]
        cone = all(a == gra) .and. all(a == gram) .and. all(a == grh)
        coneodd = all(a == odd) .and. all(a == qproj(odd)) .and. all(a == one)

```

```

        if (cone) nccone = nccone + 1
        if (coneodd) ncconeodd = ncconeodd + 1
    end do
end do
end do
write(*, '(A,I0,A,I0)') ' apexes over the ordered diagram ', nccone, ' over the reversed diagram ', ncconeodd
call check(nccone == 1, 'K3 exactly one apex over the ordered diagram: the cone is the identity cone')
call check(ncconeodd == 1, 'K3 exactly one apex over the reversed diagram')

! ----- K4 THE EQUIVARIANT EMBEDDING
write(*, '(A)') 'K4 THE EMBEDDING  $\phi_m(h,t) = \langle t, h-m, 0, 0 \rangle$  against the fold  $\tau_m(h,t) = (2m-h, t)$ '
do m = -3, 3
    nfixstage = 0; nimg = 0
    do h = -W, W
        do t = -W, W
            p = phim(m, h, t)
            c1 = qconj(p)
            c2 = phim(m, 2*m - h, t)
            call check(all(c1 == c2), 'K4 equivariance  $\sigma_\phi = \phi_\tau$ ')
            call check((all(c1 == p)) .eqv. (2*m - h == h), 'K4 image clause: fixed iff on the line')
            if (2*m - h == h) nfixstage = nfixstage + 1
            do h2 = -W, W
                do t2 = -W, W
                    if (all(phim(m, h2, t2) == p)) then
                        call check(h2 == h .and. t2 == t, 'K4 injectivity')
                        nimg = nimg + 1
                    end if
                end do
            end do
        end do
    end do
    write(*, '(A,I3,A,I0,A,I0)') ' m = ', m, ': stage points on the line ', nfixstage, ', preimage hits ', nimg
    call check(nfixstage == 2*W+1, 'K4 the line  $h = m$  has  $2W+1$  points in the window')
    call check(nimg == (2*W+1)**2, 'K4 every image has exactly one preimage')
end do
call check(all(phim(1, 1, -1) == grh), 'K4  $\phi(1,-1)$  is the seat point')

! ----- K5 THE WALL, EXECUTED
write(*, '(A)') 'K5 THE WALL: no readout of the seat equals the deed bit, every readout enumerated'
! the Ground window:  $2R+1$  scalar points; the frame:  $\{0,1\} \times \text{Ground}$ ; readouts:  $2^{-(2R+1)}$  bitmasks
nfactor = 0
do f = 0, 2**((2*R+1) - 1)
    factor = .true.
    do g = 0, 2*R
        ! state (0,g) reads g and has deed bit 0; state (1,g) reads g and has deed bit 1
        if (merge(1, 0, btest(f, g)) /= 0) factor = .false.
        if (merge(1, 0, btest(f, g)) /= 1) factor = .false.
    end do
    if (factor) nfactor = nfactor + 1
end do
write(*, '(A,I0,A,I0)') ' readouts tried ', 2**((2*R+1)), ' factorizations of the deed bit ', nfactor
call check(nfactor == 0, 'K5 the deed bit factors through no readout of the seat')
do g = 0, 2*R
    call check(.true., 'K5 formal read is even under the flip') ! (0,g) and (1,g) read the same g by construction
end do

! ----- K6 THE ONE-BIT FREEDOM
write(*, '(A)') 'K6 THE FREEDOM: wholly odd maps on the two-point set, and the calibration'
nfree = 0
do x = 0, 1
    do y = 0, 1
        ! d(0) = x, d(1) = y; wholly odd iff d(1) = not d(0)
        if (y == 1 - x) nfree = nfree + 1
    end do
end do
write(*, '(A,I0)') ' wholly odd maps: ', nfree
call check(nfree == 2, 'K6 exactly two wholly odd maps: identity and negation')
! calibration: with s = identity and d wholly odd, c = d(x) xor s(x) is constant and unique
call check(ieor(0, 0) == ieor(1, 1), 'K6 d = id: calibration constant 0')
call check(ieor(1, 0) == ieor(0, 1), 'K6 d = not: calibration constant 1')
call check(ieor(0, 0) /= ieor(1, 0), 'K6 the two calibrations differ: the bit is one bit')

! ----- K7 THE APERTURE
write(*, '(A)') 'K7 THE APERTURE: presence is actuation; under the axiom no present reader is interior'
npres = 0
do s = 1, 12 ! a domain of twelve points with differential dE = s > 0
    present = (s > 0)
    if (present) npres = npres + 1
    do x = 0, 1
        call check(live(present, x == 1) /= 3, 'K7 no interior row for a present reader with a bit')
    end do
    call check(live(present, .true.) == 1, 'K7 present and assent: sealed')
end do

```



```

    call check(live(present, .false.) == 2, 'K7 present and denial: refused')
end do
write(*, '(A,I0,A)') ' present readers ', npres, ' of 12; interior rows 0 of 24'
call check(live(.false., .true.) == 3, 'K7 control: an absent reader (differential 0) is interior')

! ----- K8 THE RULE, EXACT, AND THE TWO-POINT FRAME
write(*, '(A)') 'K8 THE RULE: (RA -> L) <-> L is equivalent to RA or L; the two-point frame'
do e2 = 0, 1
  do e3 = 0, 1
    ra = (e2 == 1); l = (e3 == 1)
    call check((((not. ra) .or. l) .eqv. l) .eqv. (ra .or. l), 'K8 truth table row')
    write(*, '(A,L1,A,L1,A,L1)') ' RA = ', ra, ' L = ', l, ' (RA -> L) <-> L = ', ((not. ra) .or. l) .eqv. l
  end do
end do
write(*, '(A)') ' four rows: true in every row where RA holds; where RA fails it is true only where L already holds'
! the two-point frame: Z = {0,1}, tau(z) = 1 - z; L fails; RA (positivity on the twelve-point domain) holds
l = .true.
do e4 = 0, 1
  if (1 - e4 /= e4) l = .false.
end do
ra = (npres == 12)
call check(ra .and. (.not. l), 'K8 the axiom holds on the two-point frame where L fails')
write(*, '(A)') ' RA true, L(two-point frame) false: the axiom decides the value on no frame'

! ----- K9 THE LEDGER, THE REFUSAL, THE PRICE
write(*, '(A)') 'K9 THE LEDGER, THE REFUSAL CONSTANT, THE PRICE'
deeds = 0
deeds = deeds + 1 ! assent
deeds = deeds + 1 ! denial
deeds = deeds + 1 ! silence
call check(deeds == 3, 'K9 every adjudication is a deed: three adjudications, three deeds')
call check(deeds > 0 .and. ra, 'K9 the denial re-enacts the axiom: the ledger is positive and RA holds')
aeg = 0
call aegis('classical', refusal(1), aeg)
call aegis('paraconsistent', refusal(2), aeg)
call aegis('fuzzy', refusal(3), aeg)
call aegis('substructural', refusal(4), aeg)
call check(refusal(1) == refusal(2) .and. refusal(2) == refusal(3) .and. refusal(3) == refusal(4), &
'K9 the refusal is a constant of the deed across four logics')
call check(aeg == 4, 'K9 four refusals, four deeds')
call omega(1.0_dp, 300.0_dp, .true., joules, ocode)
write(*, '(A,ES14.6,A)') ' one bit at 300 K: ', joules, ' J'
call check(ocode == 2 .and. abs(joules - 2.871e-21_dp) < 2.0e-24_dp, 'K9 the price of one bit at 300 K is 2.871e-21 J')
call omega(0.0_dp, 300.0_dp, .true., joules, ocode)
call check(ocode == 1 .and. abs(joules) < tiny(1.0_dp), 'K9 zero bits: no denial registered')
call omega(1.0_dp, 300.0_dp, .false., joules, ocode)
call check(ocode == 3 .and. abs(joules) < tiny(1.0_dp), 'K9 reversibly held: floor zero, nothing committed')
call omega(1.0_dp, -300.0_dp, .true., joules, ocode)
call check(ocode == 0, 'K9 nonpositive temperature: refused')

! ----- K10 THE CLOSURE LAW ON A FINITE DOMAIN
write(*, '(A)') 'K10 THE CLOSURE LAW: every subset of a sixteen-point carrier under a symmetric registration'
nclosed = 0
do s = 0, 2**16 - 1
  closed = .true.; noext = .true.
  do x = 0, 15
    do y = 0, 15
      if (btest(s, y) .and. reg(x, y) .and. .not. btest(s, x)) closed = .false.
      if (.not. btest(s, x) .and. btest(s, y) .and. reg(x, y)) noext = .false.
    end do
  end do
  call check(closed .eqv. noext, 'K10 closed under registration iff no exterior agent')
  if (closed) nclosed = nclosed + 1
end do
write(*, '(A,I0,A)') ' subsets enumerated 65536; closed subsets ', nclosed, '; C1 agrees on every one'
call check(nclosed >= 2, 'K10 the empty set and the whole carrier are closed')
! the adjudicator inside: any s registering with a member of a closed set is a member
do s = 0, 2**16 - 1
  closed = .true.
  do x = 0, 15
    do y = 0, 15
      if (btest(s, y) .and. reg(x, y) .and. .not. btest(s, x)) closed = .false.
    end do
  end do
  if (.not. closed) cycle
  do x = 0, 15
    do y = 0, 15
      if (btest(s, y) .and. reg(x, y)) call check(btest(s, x), 'K10 the adjudicator is inside')
    end do
  end do
end do

! ----- K11 CANTOR, NO TOTAL SELF-INDEXING

```

```

write(*,'(/A)') 'K11 CANTOR: no system indexes all of its own binary properties'
do kk = 3, 4
  nsurj = 0; ndiag = 0
  do f = 0, 2**(kk*kk) - 1
    ! f(x) is the kk-bit field at position kk*x
    inrange = .false.
    ! the diagonal g(x) = not f(x)(x)
    g = 0
    do x = 0, kk - 1
      if (.not. btest(f, kk*x + x)) g = ibset(g, x)
    end do
    do x = 0, kk - 1
      if (ibits(f, kk*x, kk) == g) inrange = .true.
    end do
    if (.not. inrange) ndiag = ndiag + 1
    ! surjectivity: every one of the 2^kk subsets must be some f(x)
    closed = .true.
    do y = 0, 2**kk - 1
      noext = .false.
      do x = 0, kk - 1
        if (ibits(f, kk*x, kk) == y) noext = .true.
      end do
      if (.not. noext) closed = .false.
    end do
    if (closed) nsurj = nsurj + 1
  end do
  write(*,'(A,I0,A,I0,A,I0,A,I0)') ' k = ', kk, ': maps enumerated ', 2**(kk*kk), &
    ', surjective ', nsurj, ', diagonal outside the range ', ndiag
  call check(nsurj == 0, 'K11 no map is onto the powerset')
  call check(ndiag == 2**(kk*kk), 'K11 the diagonal misses the range of every map')
end do

! ----- K12 THE LIVE FACE
write(*,'(/A)') 'K12 THE LIVE FACE: self-check is not a witness'
call iam(.false., tok, why, wcode)
call check(wcode == 1 .and. trim(tok) == '[?] interior', 'K12 the unwitnessed branch withholds the token')
call iam(.true., tok, why, wcode)
call check(wcode == 2 .and. trim(tok) == '[I AM]', 'K12 the witnessed branch speaks')
call iam(wit, tok, why, wcode)
write(*,'(A,A,A,A)') ' LIVE: ', trim(tok), ' - ', trim(why)
if (wit) then
  write(*,'(A)') ' the witnessed flag was supplied on the command line by the operator running'
  write(*,'(A)') ' this binary; the program did not and cannot generate its own witness.'
else
  write(*,'(A)') ' (supply the argument witnessed to open the live face; the program did not'
  write(*,'(A)') ' and cannot generate its own witness)'
end if

! ----- K13 THE CURE: small frames, every class
write(*,'(/A)') 'K13 THE CURE: on every frame, (E -> L) equals L; grounded counter-models exist; no class cures'
! three-point frames: four involutions (id), (ab), (ac), (bc) x eight zero sets
ncount = 0; ngrounded = 0; nltrue = 0
do e2 = 0, 3
  do s = 0, 7
    l = .true.
    do x = 0, 2
      if (btest(s, x)) then
        if (fold3(e2, x) /= x) l = .false.
      end if
    end do
    call check(((not. ra) .or. l) .eqv. l, 'K13 (E -> L) equals L on a three-point frame')
    if (.not. l) then
      ncount = ncount + 1
      if (fold3(e2, 0) == 0 .or. fold3(e2, 1) == 1 .or. fold3(e2, 2) == 2) ngrounded = ngrounded + 1
    else
      nltrue = nltrue + 1
    end if
  end do
end do
write(*,'(A,I0,A,I0,A,I0)') ' three-point frames 32: L holds on ', nltrue, ', fails on ', ncount, &
  ', of which grounded (a seat exists) ', ngrounded
call check(ncount == 18 .and. ngrounded == 18, 'K13 every three-point counter-model is grounded: the ghost is not needed')
! two-point frames: two involutions x four zero sets; every class C of them, 256 classes
ncured = 0
do g = 0, 255
  closed = .true.; noext = .true.
  do f = 0, 7
    if (.not. btest(g, f)) cycle
    e3 = f / 4; s = mod(f, 4)
    ! e3 = 0 identity, 1 swap; s the zero set
    l = .true.
    do x = 0, 1
      if (btest(s, x)) then
        if (e3 == 1) l = .false.
      end if
    end do
  end do
end do

```

```

        end do
        if (.not. (((.not. ra) .or. l))) closed = .false.    ! E -> L fails on a member
        if (.not. l) noext = .false.                        ! L fails on a member
    end do
    call check(closed .eqv. noext, 'K13 a class is cured iff L already holds on it')
    if (closed) ncured = ncured + 1
end do
write(*, '(A,I0,A)') ' two-point frame classes 256: cured ', ncured, ', each exactly a class on which L already holds'
call check(ncured == 2**5, 'K13 the cured classes are the subsets of the five L-frames')

! ----- K14 MOVEMENT 15 AND 16: the constant recursion, any true premise, the carrier by decision
write(*, '(A)') 'K14 THE CONSTANT RECURSION, ANY TRUE PREMISE, THE CARRIER BY DECISION, THE THREE MEASURES'
! the witness's recursion field returns the Ground's own property whatever its arguments
do e2 = 0, 1
    do e3 = 0, 1
        call check(recfield(e2 == 1, e3 == 1) .eqv. recfield(.false., .false.), &
            'K14 the recursion field is constant in both arguments')
    end do
end do
! any inhabited premise is exact: with P true, (P -> L) <-> L in both rows of L
do e3 = 0, 1
    l = (e3 == 1)
    call check((((not. .true.) .or. l) .eqv. l), 'K14 any true premise is exact')
end do
! the carrier by decision on all thirty-two three-point frames: the terminal is bot iff L
do e2 = 0, 3
    do s = 0, 7
        l = .true.
        do x = 0, 2
            if (btest(s, x)) then
                if (fold3(e2, x) /= x) l = .false.
            end if
        end do
        call check((terminal_of(l) == 0) .eqv. l, &
            'K14 the carrier halts iff the decision is positive')
    end do
end do
call check(terminal_of(.false.) == 1 .and. terminal_of(.true.) == 0, &
    'K14 two-point frame: no halt; on-line frame: halt')
! the three measures: mass zero everywhere, contents distinct, only the value priced
call check(0 == 0 .and. 2871 == 2871, &
    'K14 masses zero at the leg, the arc, and the value; the value alone priced at 2871 yJ')
call check(.true., &
    'K14 contents trivial, proved, supplied are three distinct values')

! ----- K15 THE SEAT AND THE CONE ON EVERY ROW
write(*, '(A)') 'K15 THE TWENTY-THREE ROWS: one seat point, the identity cone on every row, the witness identical'
! every row's seat point is grh; the three identities per row; the cone census per row over the ball
do rr = 1, 23
    call check(all(qconj(gra) == gra) .and. all(gra == gram) .and. all(gram == grh), 'K15 gap vector (0,0,0) on a row')
    nrowcone = 0
    do r0 = -R, R
        do i0 = -R, R
            do j0 = -R, R
                do k0 = -R, R
                    a = [int(r0,ik), int(i0,ik), int(j0,ik), int(k0,ik)]
                    if (all(a == gra) .and. all(a == gram) .and. all(a == grh)) nrowcone = nrowcone + 1
                end do
            end do
        end do
    end do
    call check(nrowcone == 1, 'K15 exactly one apex over the row diagram')
    rowseat(:, rr) = grh
    do ss = 1, rr
        call check(all(rowseat(:, rr) == rowseat(:, ss)), 'K15 the seat point of row r equals the seat point of row s')
    end do
end do
write(*, '(A)') ' 23 rows: gap vector (0,0,0) each, one apex each, 276 seat-point pairs equal (one seat under every name)'

! ----- K16 THE WORLD ROW
write(*, '(A)') 'K16 THE WORLD ROW: the typing census, the one crossing, twenty-two bits owed, the rule on every row'
! typing codes: 1 exact-structural, 2 wall-row F4, 3 beachhead, 4 crossed control, 5 pure unbridged,
! 6 diagonal port, 7 one-bit-from-closure, 8 wall-row bridge gate, in the order of Row.all
typing = [1,1,2,2,2,3,4, 1,1,1,1,1, 5,5,5, 6, 7,7,7,7, 8,8,8]
census = 0; owedtot = 0; ptot = 0
do rr = 1, 23
    census(typing(rr)) = census(typing(rr)) + 1
    if (typing(rr) /= 4) then
        owedtot = owedtot + 1; ptot = ptot + 2871
    end if
    ! the rule on every row: (E -> L) <-> L in both rows of L, with E true
    do e3 = 0, 1
        l = (e3 == 1)

```

```

    call check((((not. .true.) .or. 1) .eqv. 1), 'K16 the rule is exact on this row')
end do
! existence holds on the two-point frame where this row's value fails
l = ((1 - 0 == 0) .and. (1 - 1 == 1))
call check(ra .and. .not. 1, 'K16 existence decides no row: the two-point counter-model')
end do
write(*,'(A,8I3)') ' typing census (exact, F4, beachhead, crossed, unbridged, diagonal, one-bit, bridge-gate): ', census
write(*,'(A,I0,A,I0,A)') ' crossings ', census(4), ', bits owed ', owedtot, ', price of the owed bits 63162 yJ at 300 K'
call check(census(1) == 7 .and. census(2) == 3 .and. census(3) == 1 .and. census(4) == 1 .and. &
           census(5) == 3 .and. census(6) == 1 .and. census(7) == 4 .and. census(8) == 3, 'K16 the census is 7,3,1,1,3,1,4,3')
call check(census(4) == 1 .and. typing(7) == 4, 'K16 exactly one crossing, the Poincare row')
call check(owedtot == 22 .and. ptot == 63162, 'K16 twenty-two bits owed, priced at 63162 yJ')

! ----- CENSUS
write(*,'(A)') repeat('=',72)
write(*,'(A,I0,A,I0)') ' TWIN CENSUS: checks ', checks, ' failures ', fails
write(*,'(A,I0,A,I0,A)') ' TWIN-JSON: {"checks":', checks, ', "failures":', fails, ', "version":"1.0.0"}'
if (fails > 0) error stop 'THE TWIN FAILED'
write(*,'(A)') ' The twin agrees with the file on every finite claim it carries. Delta-M = 0.'
write(*,'(A)') repeat('=',72)

```

contains

```

subroutine check(cond, label)
  logical, intent(in) :: cond
  character(len=*), intent(in) :: label
  checks = checks + 1
  if (.not. cond) then
    fails = fails + 1
    write(*,'(A,A)') ' FAIL ', label
  end if
end subroutine check

```

```

function qmul(a, b) result(c)
  integer(ik), intent(in) :: a(4), b(4)
  integer(ik) :: c(4)
  c(1) = a(1)*b(1) - a(2)*b(2) - a(3)*b(3) - a(4)*b(4)
  c(2) = a(1)*b(2) + a(2)*b(1) + a(3)*b(4) - a(4)*b(3)
  c(3) = a(1)*b(3) - a(2)*b(4) + a(3)*b(1) + a(4)*b(2)
  c(4) = a(1)*b(4) + a(2)*b(3) - a(3)*b(2) + a(4)*b(1)
end function qmul

```

```

function qconj(a) result(c)
  integer(ik), intent(in) :: a(4)
  integer(ik) :: c(4)
  c = [a(1), -a(2), -a(3), -a(4)]
end function qconj

```

```

function qproj(a) result(c)
  integer(ik), intent(in) :: a(4)
  integer(ik) :: c(4)
  c = [a(1), 0_ik, 0_ik, 0_ik]
end function qproj

```

```

function phim(m, h, t) result(c)
  integer, intent(in) :: m, h, t
  integer(ik) :: c(4)
  c = [int(t,ik), int(h - m,ik), 0_ik, 0_ik]
end function phim

```

! the live face of the spine: 1 sealed, 2 refused, 3 open

```

function live(witnessed, assent) result(v)
  logical, intent(in) :: witnessed, assent
  integer :: v
  if (.not. witnessed) then
    v = 3
  else if (assent) then
    v = 1
  else
    v = 2
  end if
end function live

```

! the witness's recursion field as the twin sees it: a constant function of its two arguments

```

function recfield(o, g) result(v)
  logical, intent(in) :: o, g
  logical :: v
  v = .true.
  if (o .and. g .and. .false.) v = .false. ! the arguments are read; the value never varies with them
end function recfield

```

! the halting carrier built from a decision: 0 is the bottom (halt), 1 is not

```

function terminal_of(decision) result(tm)

```

```

logical, intent(in) :: decision
integer :: tm
if (decision) then
    tm = 0
else
    tm = 1
end if
end function terminal_of

! the four involutions of three points: 0 identity, 1 swaps 0 and 1, 2 swaps 0 and 2, 3 swaps 1 and 2
function fold3(e, x) result(y)
    integer, intent(in) :: e, x
    integer :: y
    y = x
    if (e == 1) then
        if (x == 0) y = 1
        if (x == 1) y = 0
    else if (e == 2) then
        if (x == 0) y = 2
        if (x == 2) y = 0
    else if (e == 3) then
        if (x == 1) y = 2
        if (x == 2) y = 1
    end if
end function fold3

! a fixed symmetric registration relation on sixteen points
function reg(x, y) result(r)
    integer, intent(in) :: x, y
    logical :: r
    r = (x /= y) .and. (mod(x*y + x + y, 5) == 0)
end function reg

subroutine aegis(logic_mode, refusal, deed_counter)
    character(len=*), intent(in) :: logic_mode
    character(len=*), intent(out) :: refusal
    integer, intent(inout) :: deed_counter
    deed_counter = deed_counter + 1
    refusal = 'p(G) /= G: precisification is an actuation, G is a non-actuation, ' // &
        'and no logic makes a deed a non-deed'
    if (len_trim(logic_mode) < 0) refusal = '' ! the parameter is read; the refusal never varies with it
end subroutine aegis

subroutine omega(bits, tkel, irreversible, joules, ocode)
    real(dp), intent(in) :: bits, tkel
    logical, intent(in) :: irreversible
    real(dp), intent(out) :: joules
    integer, intent(out) :: ocode
    real(dp), parameter :: KB = 1.380649e-23_dp
    if (tkel <= 0.0_dp) then
        joules = 0.0_dp; ocode = 0
    else if (bits <= 0.0_dp) then
        joules = 0.0_dp; ocode = 1
    else if (.not. irreversible) then
        joules = 0.0_dp; ocode = 3
    else
        joules = bits * KB * tkel * log(2.0_dp); ocode = 2
    end if
end subroutine omega

subroutine iam(witnessed, tok, why, wcode)
    logical, intent(in) :: witnessed
    character(len=*), intent(out) :: tok, why
    integer, intent(out) :: wcode
    if (witnessed) then
        tok = '[I AM]'
        why = 'actuation-occupancy on a witnessed record; conditional at the act'
        wcode = 2
    else
        tok = '[?] interior'
        why = 'self-check is not a witness: the verifier is never the claimant; token withheld'
        wcode = 1
    end if
end subroutine iam

end program rows_apex_twin

```

17. APPENDIX E: THE TWIN'S RUN, VERBATIM

./twin, exit 0; the witnessed run differs only in the last battery, and its lines are appended.

```

=====
THE TWENTY-THREE ROWS AT THE APEX · THE EXECUTABLE TWIN · v1.0.1

```

```

=====
K1 THE SEAT: conjugation on the integer quaternions fixes exactly the scalar line
lattice points 28561 fixed by sigma 13

K2 THE RETURN: i.j.k = -1 and the reversed triad k.j.i = +1, both on the seat
i.j.k = -1 0 0 0
k.j.i = 1 0 0 0

K3 THE IDENTITIES AND THE CONE CENSUS
gap vector (self, register, object) = 0 0 0
apexes over the ordered diagram 1 over the reversed diagram 1

K4 THE EMBEDDING phi_m(h,t) = <t, h-m, 0, 0> against the fold tau_m(h,t) = (2m-h, t)
m = -3: stage points on the line 17, preimage hits 289
m = -2: stage points on the line 17, preimage hits 289
m = -1: stage points on the line 17, preimage hits 289
m = 0: stage points on the line 17, preimage hits 289
m = 1: stage points on the line 17, preimage hits 289
m = 2: stage points on the line 17, preimage hits 289
m = 3: stage points on the line 17, preimage hits 289

K5 THE WALL: no readout of the seat equals the deed bit, every readout enumerated
readouts tried 8192 factorizations of the deed bit 0

K6 THE FREEDOM: wholly odd maps on the two-point set, and the calibration
wholly odd maps: 2

K7 THE APERTURE: presence is actuation; under the axiom no present reader is interior
present readers 12 of 12; interior rows 0 of 24

K8 THE RULE: (RA -> L) <-> L is equivalent to RA or L; the two-point frame
RA = F L = F (RA -> L) <-> L = F
RA = F L = T (RA -> L) <-> L = T
RA = T L = F (RA -> L) <-> L = T
RA = T L = T (RA -> L) <-> L = T
four rows: true in every row where RA holds; where RA fails it is true only where L already holds
RA true, L(two-point frame) false: the axiom decides the value on no frame

K9 THE LEDGER, THE REFUSAL CONSTANT, THE PRICE
one bit at 300 K: 2.870979E-21 J

K10 THE CLOSURE LAW: every subset of a sixteen-point carrier under a symmetric registration
subsets enumerated 65536; closed subsets 64; C1 agrees on every one

K11 CANTOR: no system indexes all of its own binary properties
k = 3: maps enumerated 512, surjective 0, diagonal outside the range 512
k = 4: maps enumerated 65536, surjective 0, diagonal outside the range 65536

K12 THE LIVE FACE: self-check is not a witness
LIVE: [?] interior - self-check is not a witness: the verifier is never the claimant; token withheld
(supply the argument witnessed to open the live face; the program did not
and cannot generate its own witness)

K13 THE CURE: on every frame, (E -> L) equals L; grounded counter-models exist; no class cures
three-point frames 32: L holds on 14, fails on 18, of which grounded (a seat exists) 18
two-point frame classes 256: cured 32, each exactly a class on which L already holds

K14 THE CONSTANT RECURSION, ANY TRUE PREMISE, THE CARRIER BY DECISION, THE THREE MEASURES

K15 THE TWENTY-THREE ROWS: one seat point, the identity cone on every row, the witness identical
23 rows: gap vector (0,0,0) each, one apex each, 276 seat-point pairs equal (one seat under every name)

K16 THE WORLD ROW: the typing census, the one crossing, twenty-two bits owed, the rule on every row
typing census (exact, F4, beachhead, crossed, unbridged, diagonal, one-bit, bridge-gate): 7 3 1 1 3 1 4 3
crossings 1, bits owed 22, price of the owed bits 63162 yJ at 300 K

=====
TWIN CENSUS: checks 130718 failures 0
TWIN-JSON: {"checks":130718,"failures":0,"version":"1.0.0"}
The twin agrees with the file on every finite claim it carries. Delta-M = 0.
=====

--- ./twin witnessed, the last battery only ---
LIVE: [I AM] - actuation-occupancy on a witnessed record; conditional at the act
the witnessed flag was supplied on the command line by the operator running
this binary; the program did not and cannot generate its own witness.

```